

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
COORDENAÇÃO DE ENGENHARIA ELETRÔNICA (COELE)
ENGENHARIA ELETRÔNICA

JEU TONETE
ROGÉRIO LUIS FONTANÁRIO

**APLICAÇÃO DE REDES NEURAIS MULTICAMADAS PARA
CONTROLE DE UM VEÍCULO AUTÔNOMO**

TRABALHO DE CONCLUSÃO DE CURSO

TOLEDO - PR
2016

JEU TONETE
ROGÉRIO LUIS FONTANÁRIO

**APLICAÇÃO DE REDES NEURAIS MULTICAMADAS PARA
CONTROLE DE UM VEÍCULO AUTÔNOMO**

Trabalho de Conclusão de Curso de graduação, apresentado à disciplina de Trabalho de Conclusão de Curso 2, do Curso de Engenharia Eletrônica da Universidade Tecnológica Federal do Paraná – UTFPR, como requisito parcial para obtenção do título de bacharel em Engenharia Eletrônica.

Orientadora: Jaqueline Vargas.

TOLEDO - PR
2016

RESUMO

FONTANARIO, Rogério Luis; TONETE, Jeu. **Aplicação de Redes Neurais Multicamadas Para Controle de Um Veículo Autônomo**. 2016, 55 f. Trabalho de Conclusão de Curso (Bacharelado em Engenharia Eletrônica), Universidade Tecnológica Federal do Paraná. Toledo, 2016

Uma das técnicas mais aplicadas a veículos autônomos é o uso de Inteligência Artificial pela sua capacidade de resolver problemas complexos e não-lineares. Partindo dessa base teórica o presente trabalho pretende usar o conceito de rede Múltiplas Camadas e do algoritmo de retropropagação do erro (*backpropagation*) para controlar um automodelo através de imagens. O veículo deve de forma autônoma seguir um trajeto estipulado por duas faixas na pista. Para o processamento dos dados da imagem e para controlar a direção do veículo foram utilizados um computador comum e um microcomputador *Raspberry Pi*. Utilizando dois tipos de pista e algumas técnicas para processamento de imagens, foram treinadas e testadas várias redes neurais e obtido para cada uma delas um valor percentual referente ao acerto de decisões, o qual serviu como índice de confiabilidade.

Palavras-Chave: Inteligência Artificial, Redes Neurais, Rede Multicamadas, Retropropagação de Erro, Veículos Autônomos, Segurança Automotiva.

ABSTRACT

FONTANARIO, Rogério Luis; TONETE, Jeu. **Application of a Multilayer Neural Network to control an autonomous vehicle**. 2016, 55 sheets. Final course-assignment (Electronic Engineering Bachelor Degree), Universidade Tecnológica Federal do Paraná. Toledo, 2016

One of the most applied techniques in autonomous vehicles is the use of A.I. (Artificial Intelligence) because of its capacity to solve complex and non-linear problems. Based on this theory, this work intends to use the concept of Multilayers network and the backpropagation error algorithm to control an auto model through images. The vehicle must, in an autonomous way, follow a predefined path throughout two lanes in a track. To the images data process and to control the vehicle's direction a regular computer and a microcomputer Raspberry Pi were used. Using two different types of tracks and a few techniques to the image's processing, several neural networks were trained and tested and we obtained from each one of them a percent value to the number of hits, which it was a reliability index.

Palavras-Chave: Artificial Intelligence, Neural Networks, Multilayer network, Backpropagation error, Autonomous vehicles, Automotive safety

LISTA DE ILUSTRAÇÕES

Figura 1 – Modelo de neurônio artificial.	13
Figura 2 – Representação de uma rede neural de múltiplas camadas usada neste trabalho.	15
Figura 3 – Sistema montado neste projeto.	17
Figura 4 – Tamiya TT-01 Chassis.	18
Figura 5 - Tamiya TT-01 com modificações.	18
Figura 6 – Motor de corrente contínua modelo Tamiya 540.	19
Figura 7 - Ponte H com MOSFETs tipo N.	19
Figura 8 - Funcionamento de uma Ponte H.	20
Figura 9 - Servo Motor Tamiya TSU 03.	21
Figura 10 – Tempos para controle de servos motores.	21
Figura 11 – Circuito para regulação de tensão para o projeto.	22
Figura 12 – (a) Bateria Flureon 3S 11.1V 3000mAh, (b) indicador de bateria.	23
Figura 13 – Instalação da Webcam ao automodelo.	23
Figura 14 – (a) Módulo Wireless TP-LINK TL-WN823N, (b) Roteador TP-LINK TL-WR841ND.	24
Figura 15 – <i>Raspberry Pi 2 model B+</i> , com identificação dos pinos.	25
Figura 16 – Notebook Asus Zenbook UXL303U.	26
Figura 17 – <i>Joystick XBOX ONE</i>	26
Figura 18 – Pista montada com fita adesiva.	27
Figura 19 – Pista montada com placas de papelão.	28
Figura 20 – Fluxograma do processo de criação de programas para <i>Raspberry Pi</i>	30
Figura 21 – Interface do programa Notepad++.	30
Figura 22 – Interface principal do programa PuTTY.	31
Figura 23 – Interface do programa FileZilla.	32
Figura 24 – Tela de criação de uma nova conexão no FileZilla.	33
Figura 25 – conteúdo do arquivo CMakeLists.txt usado no projeto.	33
Figura 26 – Visão geral sobre o funcionamento do programa rodando na <i>Raspberry Pi</i>	36

Figura 27 – Procedimentos executados na tarefa de captação e transmissão de imagens.....	37
Figura 28 – Procedimentos executados na tarefa de controle do motor e servo motor.	37
Figura 29 – Procedimentos executados na tarefa de monitoramento de recebimentos dos comandos.	38
Figura 30 – Ligação dos pinos <i>Raspberry Pi</i> para outros periféricos.	39
Figura 31 – Visão geral sobre o programa rodando no computador.	40
Figura 32 - Procedimentos executados no modo para coletas de dados.	41
Figura 33 - Procedimentos executados no modo para treinamento da rede.	42
Figura 34 - Procedimentos executados no modo para teste da rede neural.	43
Figura 35 – Procedimentos para modo de execução da rede neural.	45
Figura 36 – Procedimentos para o filtro detecção de faixa.	46
Figura 37 – Exemplo de imagem captada pelo sistema, filtro escala de cinza aplicado. (a) pista com fita adesiva, (b) fita com placas de papelão.	47
Figura 38 - Exemplo de imagem captada pelo sistema, filtro limiar aplicado. (a) pista com fita adesiva, (b) fita com placas de papelão.....	47
Figura 39 – Exemplo de imagem captada pelo sistema, filtro detecção de borda aplicado.	47

LISTA DE TABELAS

Tabela 1 - Parâmetros usados para treinamento da rede neural.	42
Tabela 2 – Resultados das redes neurais para pista de papelão.....	49
Tabela 3 - Resultados das redes neurais para pista de fita adesiva	49

LISTA DE SIGLAS E ACRÔNIMOS

LISTA E SIGLAS

BMW	<i>Bayerische Motoren Werke AG</i>
CC	Corrente Contínua
I2C	<i>Inter-Integrated Circuit</i>
MLP	<i>Multilayer Perceptron</i>
PWM	<i>Pulse-Width Modulation</i>
RC	Rádio Controle
RNA	Rede Neural Artificial
SPI	<i>Serial Peripheral Interface</i>
SSH	<i>Secure Shel</i>
SFTP	<i>SSH File Transfer Protocol</i>
USB	<i>Universal Serial Bus</i>

LISTA DE ACRÔNIMOS

ESC	<i>Electronic Speed Control</i>
MOSFET	<i>Metal Oxide Semiconductor Field Effect Transistor</i>
OpenCV	<i>Open Source Computer Vision</i>

SUMÁRIO

1 INTRODUÇÃO	10
1.1 JUSTIFICATIVA.....	11
1.2 OBJETIVOS.....	11
1.3 ORGANIZAÇÃO DO TRABALHO.....	12
2 REDES NEURAIS EM RECONHECIMENTO DE PADRÕES.....	13
2.1 REDE NEURAL ARTIFICIAL MÚLTIPLAS CAMADAS (<i>MULTILAYER PERCEPTRON</i>)	14
2.2 PARÂMETROS DE UMA REDE NEURAL MLP	15
3 MATERIAIS E MÉTODOS.....	17
3.1 AUTOMODELO	17
3.1.1 Motor de corrente contínua.....	19
3.1.2 Servo Motor	20
3.1.3 Placa Para Regulação de Alimentação.....	21
3.1.4 Bateria e indicador de bateria	22
3.1.5 Câmera USB.....	23
3.1.6 Sistema de Comunicação	24
3.2 <i>RASPBERRY PI</i>	24
3.3 COMPUTADOR	25
3.4 PISTAS DE TESTES	27
4 PROGRAMAS	29
4.1 PROGRAMAS PARA DESENVOLVIMENTO	29
4.2 BIBLIOTECAS USADAS.....	33
4.2.1 OpenCV	34
4.2.2 Practical C++ Sockets.....	34
4.2.3 Gamepad	35
4.2.4 PiGPIO.....	35
4.3 PROGRAMA CRIADO PARA O SISTEMA EMBARCADO	35
4.4 PROGRAMA CRIADO PARA A ESTAÇÃO FIXA	39
4.4.1 Coleta de dados.....	40
4.4.2 Treinamento.....	42
4.4.3 Testes da rede	43

4.4.4 Execução da rede neural	44
4.4.5 Filtros usados nas imagens	46
5 TESTES E RESULTADOS.....	48
6 CONSIDERAÇÕES FINAIS	51
6.1 CONCLUSÃO	51
6.2 SUGESTÕES PARA TRABALHOS FUTUROS	52
REFERÊNCIAS.....	53
ANEXO A – PONTE H.....	55

1 INTRODUÇÃO

Nos últimos anos várias empresas tanto do setor de tecnologia quanto no setor automobilístico trouxeram para seus investimentos o tema veículos autônomos. Montadoras gigantes como Mercedes, BMW, Audi, Volkswagen, Volvo, General Motors e Tesla, assim como companhias conhecidas no ramo de inovações, como a Google, começaram a apresentar seus protótipos de automóveis que têm a capacidade de pilotar sozinhos, necessitando apenas que o condutor humano dê um comando ou simplesmente estipule um trajeto¹.

A Google, por exemplo, com seu projeto “*Self-Driving Car Project*” está desenvolvendo um piloto automático capaz de dirigir na cidade, com reconhecimento de semáforos, cruzamentos e placas^{1,2}. Já a Mercedes tem um sistema onde é utilizado o reconhecimento das bordas da pista, o qual identifica quando o condutor está saindo da faixa e atua vibrando o volante, além de poder acionar os freios de um dos lados do veículo de modo a ajudar na realização da curva^{1,3}. A “novata” Tesla aplica conceitos parecidos, possuindo sistemas para o veículo mudar de faixa, seguir linhas e estacionar sozinho⁴. Esta tecnologia de carros autônomos também pode ser desenvolvida utilizando algoritmos de aprendizagem de máquina supervisionados, onde o algoritmo faz previsões com base em conjuntos de exemplos identificando padrões de comportamento.

Considerando uma pista simples, sem cruzamentos ou semáforos, com apenas duas linhas de borda, este trabalho visa desenvolver um automodelo controlado por uma rede neural, que utilizando de imagens captadas por uma câmera, consiga controlar a direção (por meio de um servo motor) de modo que o veículo se mantenha dentro das bordas de uma pista.

O controle da direção é realizado por meio de uma rede neural artificial (RNA) de múltiplas camadas. As imagens são captadas por uma câmera e pré-processada por um minicomputador *Raspberry Pi*, e na sequência as transmite para um computador através de um sistema de comunicação veículo-computador. Esses dados são usados para o treinamento da rede e depois para a execução da mesma. Em modo de execução da rede neural, o computador funciona como um acelerador de hardware, ou seja, devido a quantidade de cálculos necessários, foi utilizado um processador mais potente que o presente no *Raspberry Pi* de modo a ganhar

velocidade no processo. Assim, espera-se que após realizar algumas voltas supervisionadas no trajeto proposto, onde os dados de entrada e controle são definidos manualmente, seja possível capturar dados suficientes para treinar a rede neural. Após o treinamento da mesma, os resultados devem ser semelhantes ao controle manual.

1.1 JUSTIFICATIVA

Um dos principais fatores causadores de óbitos no mundo são os acidentes de automóveis⁵, que por sua vez são em maioria causados por erro do condutor^{6,7}. Além disso, os gastos com acidentes estão estimados em bilhões⁷. Com o intuito de reduzir esses números, várias empresas do setor automobilístico vêm desenvolvendo sistemas de assistência autônoma.

Além de melhorar a segurança nas estradas, esses sistemas também possibilitam que pessoas com capacidades reduzidas, como deficientes e idosos, tenham a possibilidade de se locomover com o automóvel. Outro benefício é um possível aproveitamento do tempo usado para dirigir, que poderá ser voltado para outras tarefas, como trabalho, estudo, recreação e descanso⁷.

Para o desenvolvimento de veículos autônomos, um dos recursos mais utilizados é a Inteligência Artificial, pela sua capacidade de resolver problemas complexos envolvendo processamento de imagem, reconhecimento de padrões, e outros problemas não lineares⁸.

Tendo em vista as melhorias que veículos autônomos podem trazer em um futuro próximo, decidiu-se realizar um trabalho de pesquisa sobre o referido assunto, onde pretende-se usar redes neurais para o controle autônomo de um veículo, onde se planeja desenvolver um veículo que trafegue por uma pista de testes utilizando técnicas de redes neurais para o controle de direção do mesmo.

1.2 OBJETIVOS

O objetivo deste trabalho é desenvolver um veículo autônomo que trafegue por uma pista de testes utilizando técnicas de redes neurais para o controle do mesmo. Como entrada de uma RNA será utilizado imagens captadas e pré-processadas, já

como saída, o comando da direção. A execução do projeto terá foco nos seguintes objetivos específicos:

- a) Definição das estruturas do projeto, tais como a pista de testes, o veículo e seus sistemas de controle e comunicação; além dos hardwares que executarão o processamento de imagem, treinamento e execução da rede.
- b) Coleta, análise e filtragem manual de dados para o treinamento da rede neural.
- c) Desenvolvimento da rede neural.
- d) Testes de desempenho da rede criada, e possíveis melhorias no sistema.

1.3 ORGANIZAÇÃO DO TRABALHO

Este trabalho está apresentado em 5 capítulos, onde o Capítulo 1 apresenta uma pequena explicação da motivação e importância da realização deste trabalho no âmbito da eletrônica e da tecnologia, além dos possíveis benefícios que um projeto nessa área pode trazer.

O Capítulo 2 objetiva realizar uma rápida revisão teórica a respeito do estudo de Redes Neurais, focando na explicação da Rede Multicamadas.

O Capítulo 3 apresenta os materiais empregados, programas e bibliotecas utilizados e a metodologia que foi aplicada para a construção de um automodelo, seu sistema de comunicação, sua maneira de coleta de dados, treinamento e execução de uma rede neural.

No Capítulo 5 são apresentados os resultados de várias redes neurais para diferentes tratamentos de imagem, e a validação das mesmas.

Por fim, o Capítulo 6 se destina a algumas observações e considerações finais sobre o desempenho do projeto, dificuldades e ideias de possíveis melhorias para futuros estudos.

2 REDES NEURAIS EM RECONHECIMENTO DE PADRÕES

Para melhor compreensão do projeto a ser desenvolvido, faz-se necessário apresentar a importância da técnica de processamento a ser utilizada. Tal técnica, denominada Redes Neurais, visa detectar padrões treinados utilizando amostras previamente coletadas. Neste trabalho, será necessário coletar amostras das imagens da estrada, para que, quando executado o algoritmo, o veículo saiba identificar as características (curva ou reta).

Uma Rede Neural Artificial (RNA) é uma máquina projetada para resolução de problemas, visando modelar a maneira como o cérebro realiza uma tarefa particular ou função de interesse⁸. Tem como base o neurônio artificial (Figura 1), no qual apresenta uma saída, sendo essa a soma dos valores de suas múltiplas entradas, visivelmente inspirado nos dendritos dos neurônios biológicos. Os pesos simulam as sinapses e o bias é uma forma de controlar a ativação do neurônio. O treinamento da RNA constitui-se basicamente da adaptação dos pesos sinápticos de forma ordenada e respeitando à algoritmos pré-definidos, fazendo com que responda corretamente aos sinais de entrada.

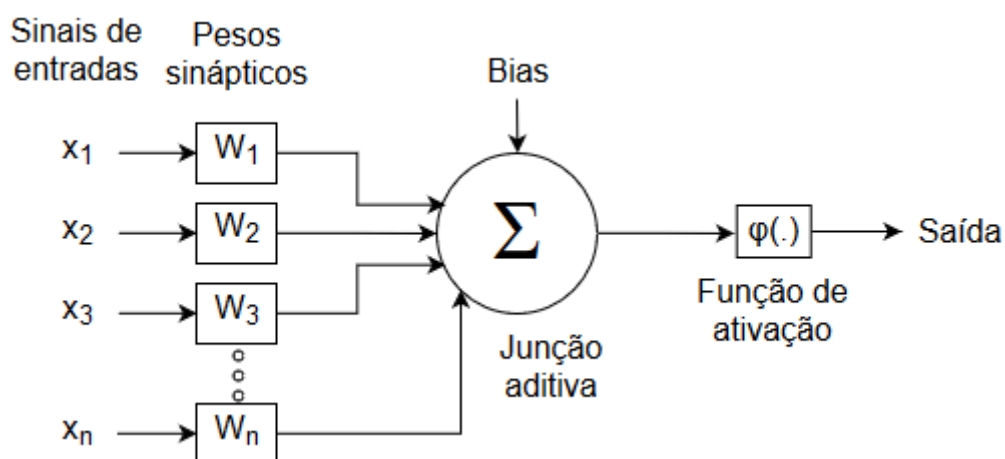


Figura 1 – Modelo de neurônio artificial.

Fonte: Autoria Própria.

Além disso, a rede apresentada na Figura 1 é uma rede supervisionada, ou seja, a saída é comparada com uma resposta desejada (supervisor), garantindo que a RNA atingiu o valor correto. Na literatura podem ser encontrados problemas que

utilizam redes de camada simples e redes de múltiplas camadas⁹. Devido a não-linearidade do problema apresentado neste trabalho, será utilizado uma RNA de múltiplas camadas.

2.1 REDE NEURAL ARTIFICIAL MÚLTIPLAS CAMADAS (*MULTILAYER PERCEPTRON*)

Uma RNA de múltiplas camadas (também conhecida como *Multilayer Perceptron* - MLP) utiliza múltiplas entradas, uma ou mais camadas ocultas e uma camada de saída⁸. Para a o treinamento da mesma será usado o algoritmo de retropropagação de erro (*Backpropagation*), que possui este nome pois utiliza o erro da última camada para calcular o erro da camada anterior, sucessivamente, até chegar na camada de entrada. O algoritmo é apresentado pelos seguintes passos:

1. Inicializar os parâmetros da rede (pesos e bias) e o valor da taxa de aprendizado;
2. Submeter, a cada iteração, todos os vetores entrada-saída do conjunto de treinamento à rede, ajustando os parâmetros da seguinte forma:
 - a. Fase à frente:
 - i. Aplicar o padrão de entrada à camada de entrada da rede;
 - ii. Para cada camada, processar o sinal de entrada e calcular a saída de seus neurônios;
 - iii. Calcular os erros produzidos na camada de saída da rede.
 - b. Fase atrás:
 - i. Atualizar os pesos da camada de saída;;
 - ii. Para todas as camadas internas da rede, atualizar os pesos a partir da estimativa gerada pela retropropagação dos erros. O erro de um determinado neurônio de uma camada interna é calculado utilizando os erros dos neurônios da camada seguinte conectados a ele, ponderados pelos pesos relacionados a estas conexões.
3. Repetir o passo 2 até que a função de custo (Erro Quadrático Médio) tenha seu valor minimizado ou até que se tenha completado um determinado número de ciclos de treinamento.

O uso de camadas ocultas na rede multicamadas implica na capacidade de abstrair as características dos vetores de entrada, o que possibilita o trabalho com sistemas não lineares, como o reconhecimento de padrões. Considerando que este trabalho pretende usar as imagens como sensores para o veículo autônomo, é possível observar na Figura 2, como será relacionado uma imagem e seus pixels com as entradas da rede e sua propagação à saída. A imagem tem tamanho 160x60 pixels, ou seja, cada pixel da imagem é uma entrada da rede, e neste caso, 9600 entradas. Além disso, definido assim que a saída possui cinco níveis discretos de direção indicando ângulo para o veículo, que nomeamos como totalmente à esquerda, esquerda, frente, direita e totalmente à direita.

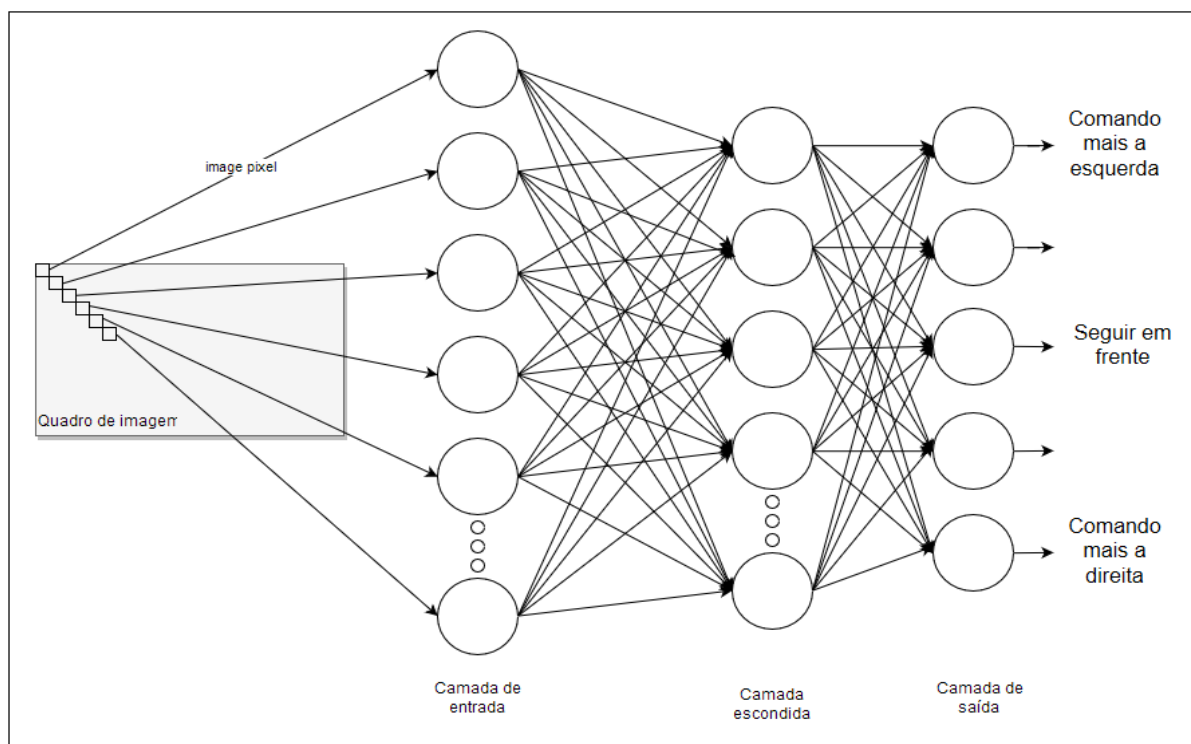


Figura 2 – Representação de uma rede neural de múltiplas camadas usada neste trabalho.

Fonte: Autoria Própria.

2.2 PARÂMETROS DE UMA REDE NEURAL MLP

Uma grande dificuldade que se encontra no treinamento de RNAs do tipo MLP utilizando o algoritmo de *backpropagation* é a configuração dos parâmetros de treinamento como, a quantidade de camadas ocultas, número de neurônios em cada

camada, taxa de aprendizado e *momentum*. A escolha desses parâmetros é um processo ainda pouco compreendido e que conta com poucos estudos¹⁰.

O objetivo do treinamento é obter um modelo não tão rígido que não possa modelar fielmente os dados e nem tão flexível à ponto de modelar o ruído presente neles¹⁰. Muitos programas já empregam algoritmos de treinamento de MLPs por *backpropagation*, ficando a cargo do desenvolvedor da RNA em questão, escolher os parâmetros desse treinamento.

Como dito anteriormente, o uso de camadas ocultas nas MLPs implica na capacidade de abstrair as características dos vetores de entrada e assim trabalhar com reconhecimento de padrões. Mas é aconselhável, que o número de camadas ocultas não deva ser muito grande, pois ao utilizar o erro médio durante o treinamento para atualizar os pesos das sinapses da camada imediatamente anterior, ele se torna cada vez menos útil ou preciso, pois a cada nova camada, o erro acaba sendo uma estimativa sobre o erro anterior. Para a maioria dos problemas nos quais RNA são usadas, como classificação e estimação, uma camada é suficiente para gerar um modelo bom, sendo necessário em poucos casos, o uso de duas camadas ocultas¹⁰.

O número de neurônios em cada camada oculta geralmente é definido empiricamente, de modo que não seja grande demais que faça a RNA decorar ao invés de aprender e nem tão pequeno que faça ela perder muito tempo tentando encontrar um modelo ótimo.

A taxa de Aprendizado é responsável por manter a estabilidade do processo de minimização do erro, evitando desse jeito que haja uma variação muito brusca na correção dele a cada iteração, ou seja, utilizando uma taxa pequena o sistema necessita de mais iterações para atingir o ponto ótimo, mas evita assim que ocorra instabilidades ao processo¹¹. Esse valor varia de 0,1 a 1 e em alguns programas essa variável pode ser adaptativa, e varia conforme a execução do treinamento. já em outros, esse valor pode ser fixo, e por isso recomendasse o uso de um valor baixo¹⁰.

O momentum permite à rede realizar ajustes de peso razoavelmente grande enquanto as correções forem na mesma direção para vários padrões, e uma pequena taxa de aprendizado para quaisquer erros de padrão de treinamento. O resultado da aplicação deste parâmetro ao treinamento é reduzir o tempo de treinamento.

3 MATERIAIS E MÉTODOS

A metodologia que foi empregada neste projeto tem como base o diagrama representado pela Figura 3, que indica os materiais necessários e suas interconexões. Nos próximos tópicos os recursos presentes nesse diagrama serão apresentados.

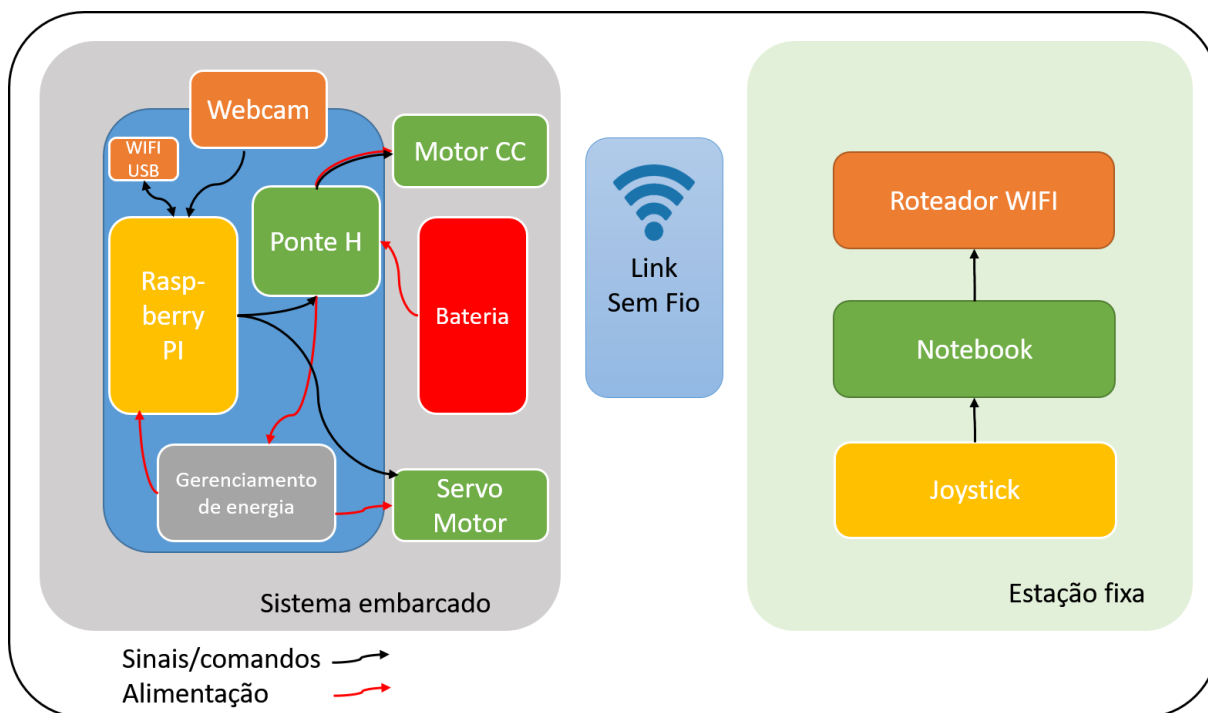


Figura 3 – Sistema montado neste projeto.

Fonte: Autoria própria.

3.1 AUTOMODELO

Para a execução do projeto pretendido foi adquirido um automodelo WRC TT-01E da marca Tamiya (Figura 4). O mesmo possui tração nas quatro rodas através de um único motor de corrente contínua, controlado por um circuito de controle de velocidade eletrônico para motores com escova (do inglês *Electronic Speed Control - ESC BRUSHED*). A direção é feita por meio de um servo motor. Os dois sistemas são controlados pelo circuito receptor que faz a comunicação com um rádio controle (RC) de 2.4GHz e envia sinais de comando para a ESC e para o Servo.

Para obter um melhor acesso ao sistema embarcado sobre o projeto, os sistemas de comunicação e atuação originais foram substituídos por circuitos especialmente projetado para atender melhor as demandas de controle. Assim foi dividido o trabalho de comunicação e atuação em outros circuitos eletrônicos.

O novo sistema de controle de veículo conta com uma placa *Raspberry Pi*, um modulo Wifi-USB, uma câmera USB, uma Ponte H, uma placa para regulação de tensão de alimentação e gerenciamento de energia, bateria de 12 volts e um indicador nível de bateria.

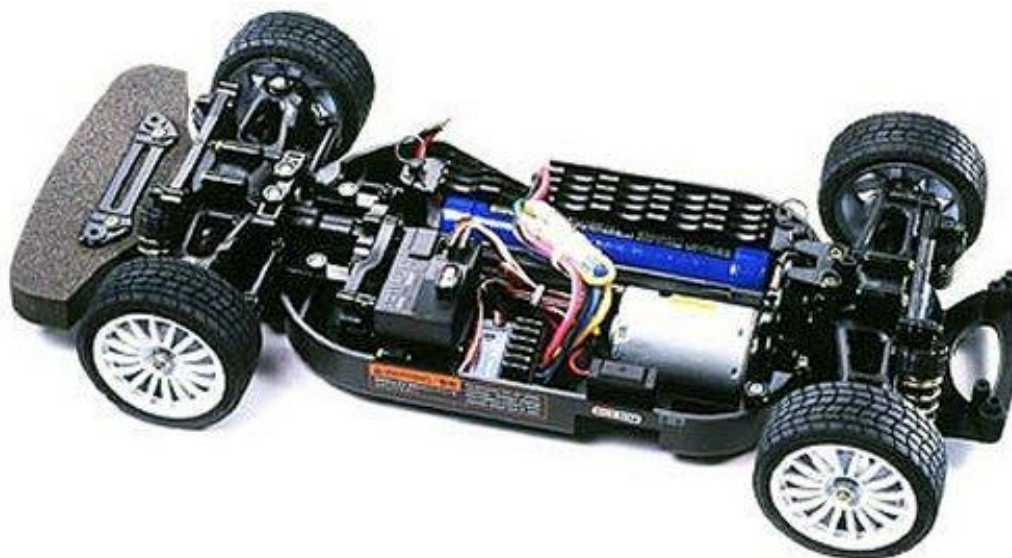


Figura 4 – Tamiya TT-01 Chassis.

Fonte: <http://www.rcscrapyard.net/tamiya-ford-escort-wrc.html>

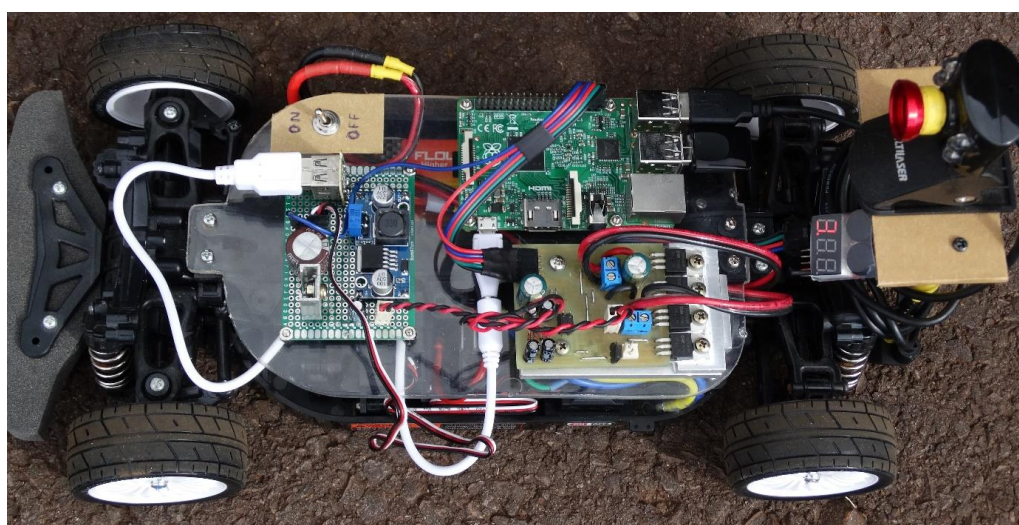


Figura 5 - Tamiya TT-01 com modificações.

Fonte: Autoria própria.

3.1.1 Motor de corrente contínua

O automodelo utiliza para a locomoção um motor de corrente contínua (motor CC) com escovas modelo Tamiya 540 (Figura 6). O controle de um motor CC é simples, para mudar o sentido de rotação, basta mudar o sentido da corrente, para variar a velocidade, varia-se a tensão média utilizando PWM (modulação por largura de pulso).

Originalmente, o automodelo utilizava um controlador de velocidade eletrônico (ESC) que foi substituído devido à limitação quanto a alimentação, pois o mesmo era indicado para o uso de baterias de 7,4 volts o que o tornou incompatível com os modelos de bateria de 12 volts adquiridas para o projeto. Por esse motivo foi projetado uma Ponte H mais robusta (Figura 7), a qual permite uma faixa de valores de tensão para a alimentação maior.



Figura 6 – Motor de corrente contínua modelo Tamiya 540.

Fonte: Autoria própria.

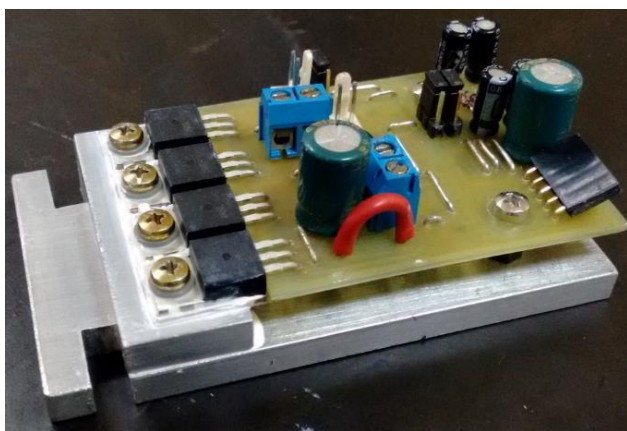


Figura 7 - Ponte H com MOSFETs tipo N.

Fonte: Autoria Própria.

A Ponte H é um circuito para o controle do sentido de corrente e valor de tensão a partir de uma fonte fixa e é muito utilizado para o controle de motores CC, onde pode se variar a direção e a velocidade do mesmo. O modelo básico é constituído por 4 chaves e o acionamento correto das chaves permite alternar o sentido da corrente aplicada aos terminais de entrada de tensão do motor. O diagrama da Figura 8 exemplifica o funcionamento. Aplicando ainda a técnica de modulação por largura de pulso (PWM), pode se então variar a tensão média que liga o motor, alterando sua velocidade.

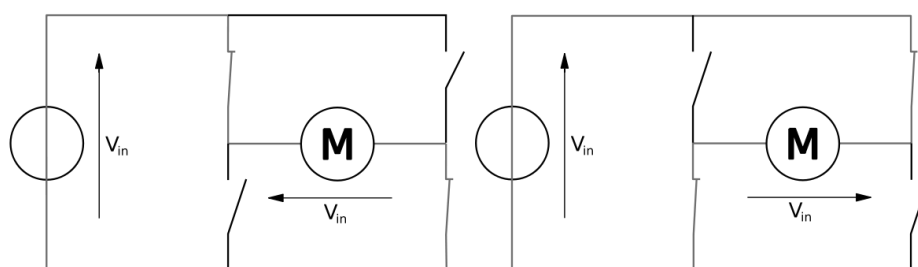


Figura 8 - Funcionamento de uma Ponte H.

Fonte: <http://www.arduinoecia.com.br/2014/04/control-de-motor-cc-com-o-l293d-ponte-h.html>

Assim o controle de velocidade do Motor CC foi dividido entre a Ponte H e a *Raspberry Pi* que controla quais chaves são acionadas e o PWM delas.

O circuito da Ponte H foi feito para o acionamento de quatro MOSFETs do tipo N, na qual se usa um circuito dobrador de tensão para o acionamento das chaves da parte de alta, onde é necessária uma tensão na Porta (*Gate*) em torno de 4 volts maior que a de Dreno (*Drain*), que quando acionado será a mesma da alimentação. O projeto é para o uso genérico de MOSFETs do tipo N, ou seja, as limitações de corrente são referentes aos MOSFETs utilizados, neste caso, o IRF640N, onde a tensão e corrente máxima são em torno de 200 V e 18 A. O esquemático do projeto se encontra na seção de anexos.

3.1.2 Servo Motor

O sistema de direção é feito através de um Servo Motor Tamiya TSU 03 (Figura 9). O funcionamento do servo se dá através de um sinal de comando,

geralmente usando PWM, onde se mantem uma largura de pulso de 1 ms para a angulação 0° e uma largura de pulso de 2 ms para a angulação de 180° , assim se faz a variação de ângulo variando a largura de pulso entre 1ms e 2 ms (Figura 10). No caso do veículo utilizado no trabalho, utilizou-se uma biblioteca onde a frequência de PWM é fixa em 50 Hz e nas funções de comando são enviados o tempo em milissegundos.



Figura 9 - Servo Motor Tamiya TSU 03.

Fonte: Autoria própria.

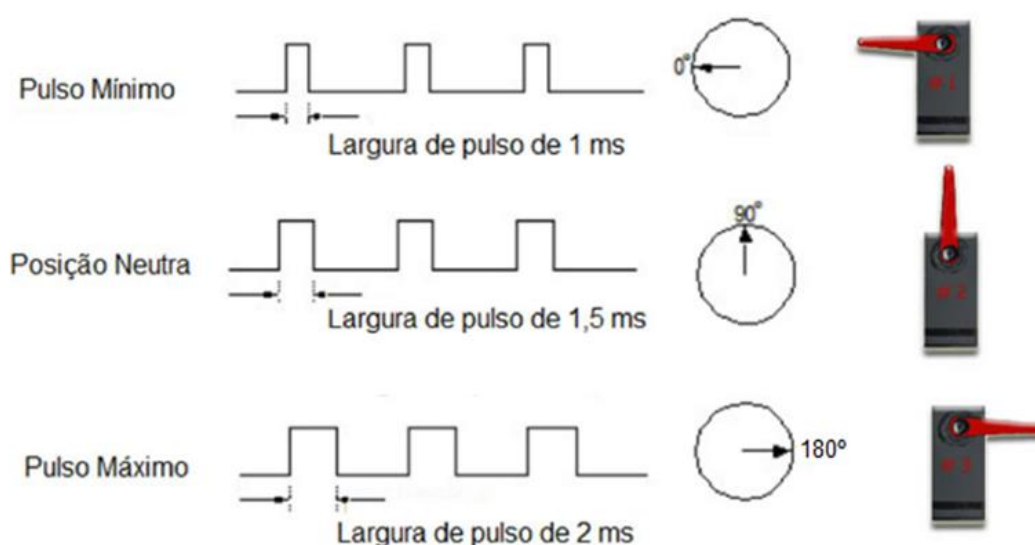


Figura 10 – Tempos para controle de servos motores.

Fonte: <http://arduinoescola.blogspot.com.br/2015/03/controlando-servo-motores-em-scratch.html>

3.1.3 Placa Para Regulação de Alimentação

O automodelo utiliza uma bateria de 12 Volts como fonte de energia, mas como eram necessários 5 Volts para alimentar o *Raspberry Pi* e de 4.8 a 6 Volts para

o servo motor, foi feito uma placa para essa adequação de tensão. A bateria é conectada na Ponte H e de lá ramificasse para uma placa que utiliza um conversor CC-CC do tipo Buck, para regular a tensão de saída em 5 Volts em um conector USB, e um regulador de tensão baseado no CI 7805 para fornecer alimentação para o Servo em um conector adequado ao mesmo (barra de pinos).

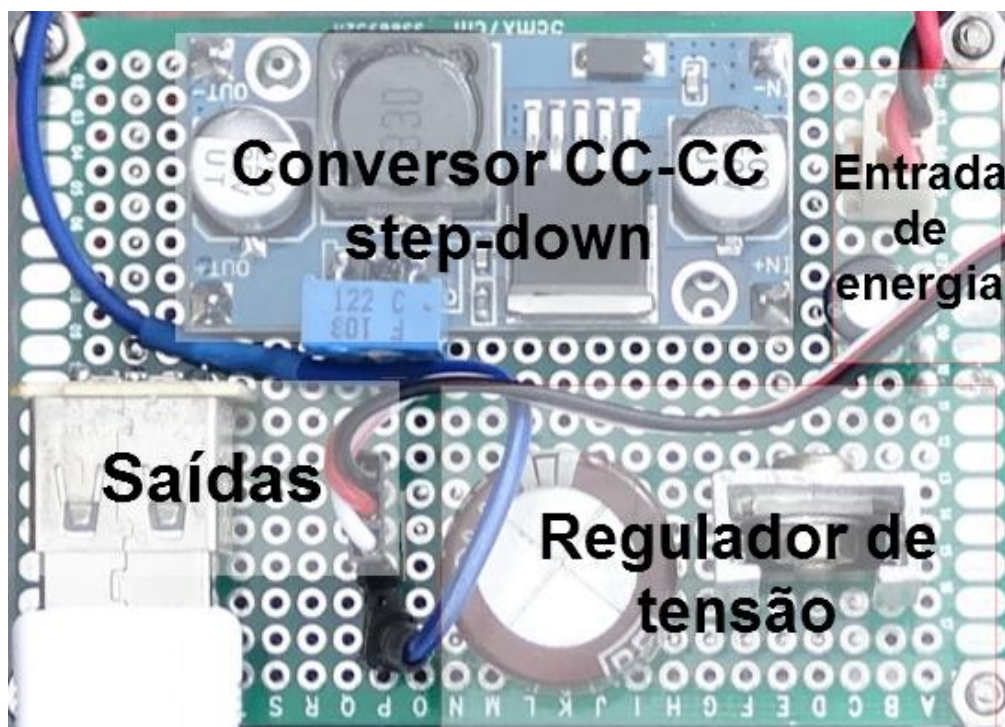


Figura 11 – Circuito para regulação de tensão para o projeto.

Fonte: Autoria própria.

3.1.4 Bateria e indicador de bateria

A alimentação do veículo se dá através de baterias de Polímero de Lítio Flureon de 11.1V 3000mAh (Figura 12 (a)) e foram utilizadas em conjunto com um indicador de bateria com alarme (Figura 12 (b)), o qual fornece a indicação do nível de cada célula da bateria e a tensão total, além de ligar um sistema de *buzzers* quando alguma célula chega em nível crítico de tensão.

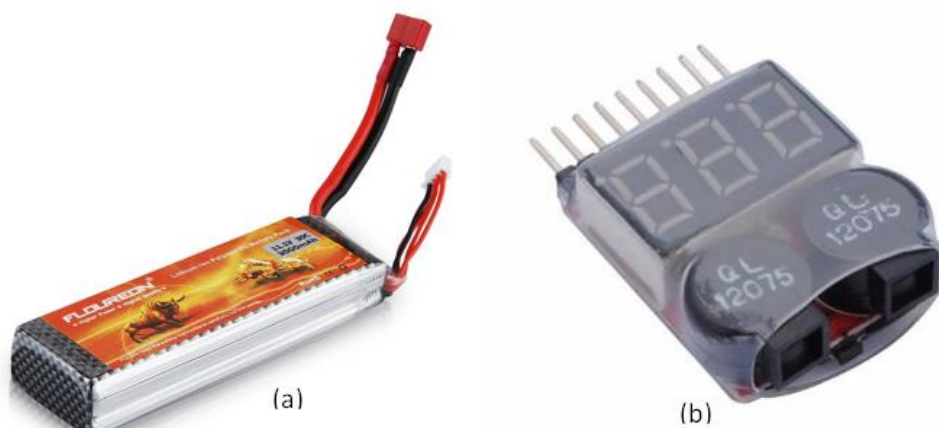


Figura 12 – (a) Bateria Flureon 3S 11.1V 3000mAh, (b) indicador de bateria.

Fonte: Autoria própria.

3.1.5 Câmera USB

Para a captação de imagens usou-se uma Webcam USB MultiLaser WC040 (Figura 13) com a adaptação de uma lente para aumentar o ângulo de visão da mesma, possibilitando a captação de imagens mais amplas. A câmera foi posicionada na parte traseira do veículo (Figura 14) de modo que a mesma consiga captar um pedaço da parte da frente do automodelo e as faixas usadas como pista.



Figura 13 – Instalação da Webcam ao automodelo.

Fonte: Autoria própria.

3.1.6 Sistema de Comunicação

Para a comunicação do sistema, resolveu-se fazer a comunicação do computador com a *Raspberry Pi* através de uma rede sem fio e o protocolo UDP (User Datagram Protocol), para isso utilizou-se um adaptador *Wireless USB TP-LINK TL-WN823N* (Figura 14 a) com velocidade de até 300Mbps ligado à *Raspberry Pi* e um Roteador TP-LINK TL-WR841ND (Figura 14 b) também com 300Mbps, gerando uma rede local conectando o notebook e a *Raspberry Pi*.

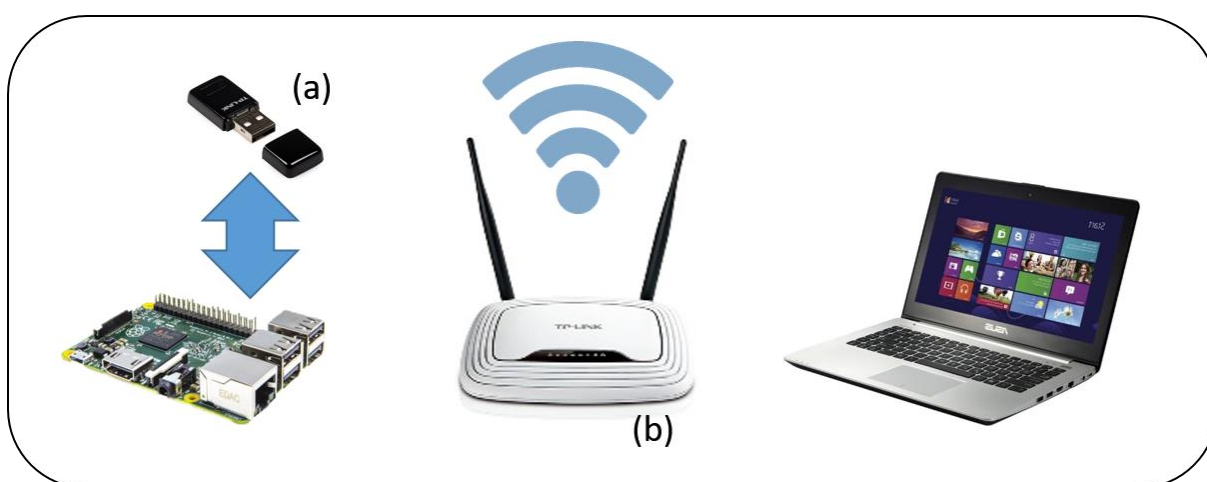


Figura 14 – (a) Módulo Wireless TP-LINK TL-WN823N, (b) Roteador TP-LINK TL-WR841ND.

Fonte: Autoria própria.

3.2 RASPBERRY PI

Raspberry Pi (Figura 15) é um minicomputador de 1 GHz de processamento com um hardware integrado em uma única placa, nela é possível conectar periféricos com conexão USB além de possuir protocolos de comunicação como SPI, I2C entre outras. Por conta desses recursos, a *Raspberry Pi* foi instalada sobre o veículo para acessar uma câmera com conexão USB, um módulo WiFi e controlar a Ponte H e um Servo Motor.

O sistema operacional embarcado no minicomputador é o *Raspbian Jessie*, que é uma variante do Debian otimizada para o conjunto de instruções do hardware da *Raspberry Pi*.

A *Raspberry Pi* recebe informações pelo módulo WiFi referentes ao controle de aceleração e direção. Além disso, ela faz um pré-processamento de imagem captada, transformando-a em escala de cinza e ajustando seu tamanho para posterior envio.

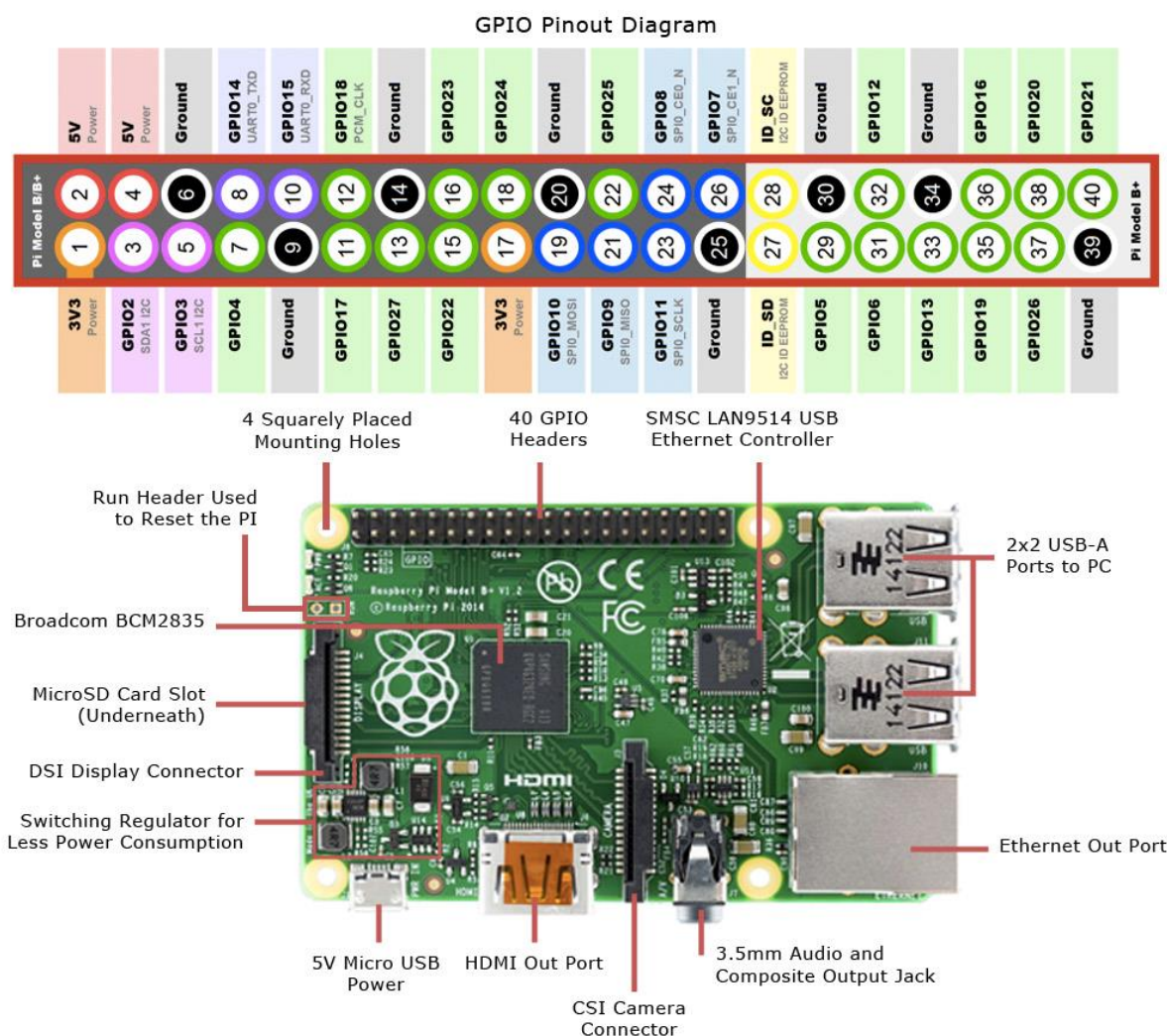


Figura 15 – *Raspberry Pi 2 model B+*, com identificação dos pinos.

Fonte: <http://raspberrypi.org/>

3.3 COMPUTADOR

O computador usado para este projeto é um Notebook da marca Asus, modelo UXL303U (Figura 16). Ele conta com um processador Intel i7 5500U de até 2.9GHz, memória RAM 8GB rodando o sistema operacional Microsoft Windows 10. O notebook roda o programa que recebe via protocolo UDP as imagens geradas pela *Raspberry Pi*, aplica a rede neural e retorna via UDP os comandos de direção e aceleração para

o minicomputador. Outras funções que auxiliam o manuseio do automodelo e também realizam o treinamento da rede neural foram implementadas.



Figura 16 – Notebook Asus Zenbook UXL303U.

Fonte: <http://www.asus.com/zenbook/ca-en/>

Ao computador foi conectado um *joystick* USB do modelo Xbox One (Figura 17), responsável pela interface com o usuário do sistema. Foi utilizado deste *joystick* o direcional analógico esquerdo, o *trigger* analógico direito e um de seus botões.



Figura 17 – Joystick XBOX ONE.

Fonte: <http://www.xbox.com/pt-BR/xbox-one/accessories/controllers/wireless-controller>

3.4 PISTAS DE TESTES

Para os treinamentos e testes do projeto fez-se necessário um ambiente controlado com boa iluminação e espaço amplo, espaço este encontrado no ginásio de esportes da Universidade Tecnológica Federal do Paraná. Neste ambiente foram criadas duas pistas, cada uma contendo duas curvas e alguns trechos de retas, mas constituídas de materiais diferentes. A primeira pista utiliza uma fita adesiva colada no chão (Figura 18) enquanto a segunda pista utiliza blocos de papelão montados sem nenhum tipo de fixação (Figura 19), possibilitando formar pistas com larguras diferentes. Ambos os materiais fornecem marcações de cor clara, o que facilita a identificação das bordas por alguns algoritmos de pré-processamento.



Figura 18 – Pista montada com fita adesiva.

Fonte: Autoria própria.



Figura 19 – Pista montada com placas de papelão.

Fonte: Autoria própria.

4 PROGRAMAS

Neste tópico serão descritos tanto os programas usados para o desenvolvimento deste trabalho, quanto as bibliotecas usadas e o funcionamento dos programas criados para o sistema embarcado e a estação fixa. Os códigos produzidos estão disponíveis para visualização e compartilhamento pelo *link*¹² disponível na seção de referências.

4.1 PROGRAMAS PARA DESENVOLVIMENTO

Alguns programas e ferramentas foram essenciais para a realização do projeto. Será descrito a seguir os principais programas que auxiliaram tanto o desenvolvimento dos códigos quanto sua transferência para a *Raspberry Pi* e sua posterior compilação, sendo eles: Notepad++, Microsoft Visual Studio Community, PuTTY, Filezilla e Cmake.

Para este trabalho, o código que roda no notebook em ambiente Windows foi feito utilizando o ambiente de Desenvolvimento Integrado (IDE) Microsoft Visual Studio Community 2015. Esta versão do Microsoft Visual Studio é uma ferramenta completa, distribuída gratuitamente, tendo foco no desenvolvimento de aplicativos não corporativos.

Já o código fonte criado para embarcar a *Raspberry Pi* foi desenvolvido em um editor de texto simples para Windows e transferido para a *Raspberry Pi*. Após isso foi gerado os arquivos para compilação e gerado o executável. O fluxograma apresentado na Figura 18 demonstra como é este processo.

O código para a Raspberry foi escrito no editor de texto e códigos *Notepad++* em ambiente *Windows* (Figura 19). Ele é distribuído por uma licença de código livre e tem suporte a várias linguagens de programação, inclusive a usada para os códigos desse projeto. O *Notepad++* fornece uma interface limpa, compacta e cheia de recursos para agilizar o processo de criação de códigos, como suporte a guias, auto completar, destaques para palavras reservadas e divisão da tela para visualizar diversos arquivos ao mesmo tempo.

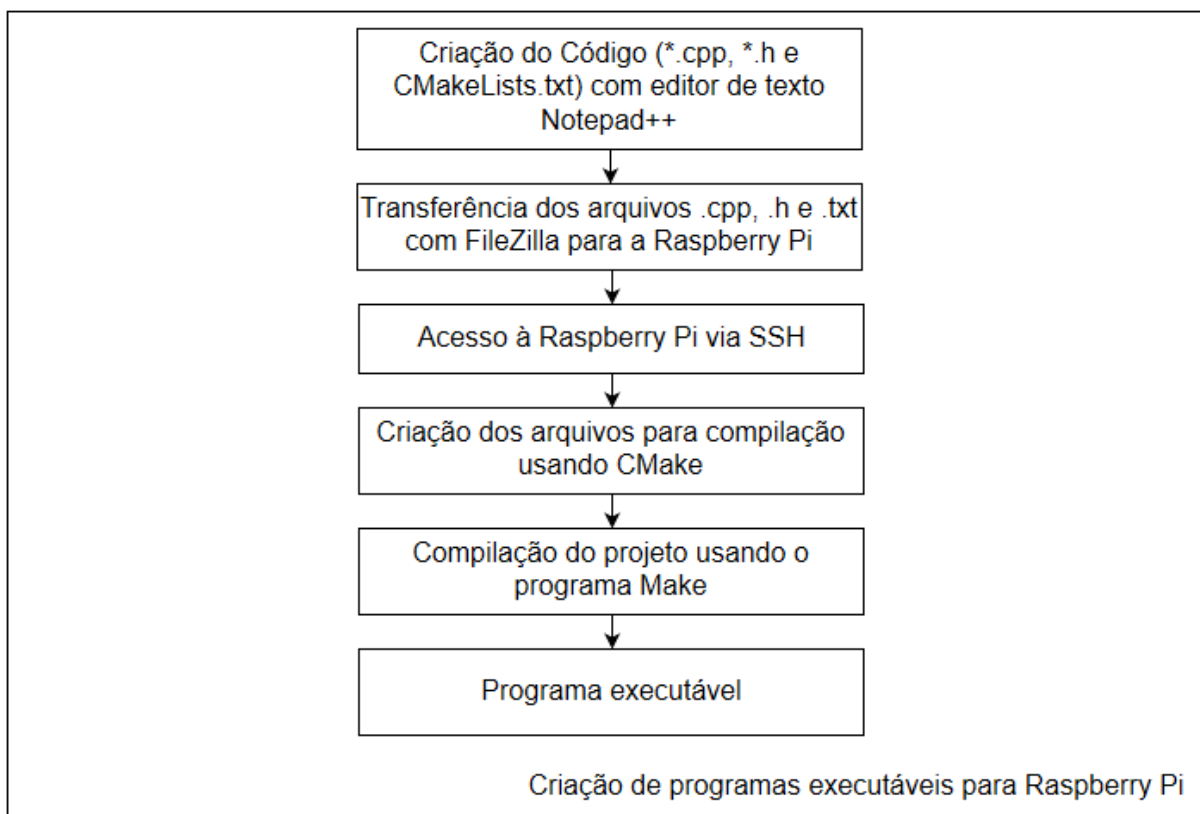


Figura 20 – Fluxograma do processo de criação de programas para *Raspberry Pi*.

Fonte: Autoria própria.

```

//UDPClient.cpp
72 //gpioSetPWMPinFrequency(PIN10, 50);
73
74
75 //int bytesRcvd;
76 while(1){
77     sock.recvFrom(recvData, BUFSIZE, sourceAddress, sourcePort);
78
79     if((int)recvData[0] > 0){//frente
80         gpioWrite(Q2, 0);
81         gpioPWM(Q3, 0);
82         gpioWrite(Q4, 1);
83         gpioPWM(Q1, (int)recvData[0]);
84         cout << (int)recvData[0]<<" frente "<<endl;
85     }else if((int)recvData[0]==0 && (int)recvData[1]>127){
86         gpioWrite(Q4, 0);
87         gpioPWM(Q1, 0);
88         gpioWrite(Q2, 1);
89         gpioPWM(Q3, (int)recvData[1]);
90         cout << (int)recvData[1]<<" tras "<<endl;
91     }else{
92         gpioWrite(Q2, 0);
93         gpioPWM(Q1, 0);
94         gpioWrite(Q4, 0);
95         gpioPWM(Q3, 0);
96         //cout <<" parado ";
97     }
98
99     if(recvData[2] != servotemp){
100         gpioServo(SERVO, (int)map(recvData[2],0,255,1000,2000));
101         servotemp=recvData[2];
102         cout << map(recvData[2],0,255,1000,2000)<<endl;
103     }
104     //cout << (int)recvData[0] << " " << (int)recvData[1] << " " << (int)recvData[2] << endl;
105 }
106 catch (SocketException se) {
107     cerr << se.what() << endl;
108     exit(1);
109 }
110 gpioTerminate();
111 return 0;
112
113
//autonomous.cpp
79 long scale(long x, long in_min, long in_max, long out_min, long out_max){
80     return (x-in_min)*(out_max-out_min)/(in_max-in_min)+out_min;
81 }
82
83
84 void *recv timer(void *arg){
85     clock_t next_cycle, last_cycle;
86     double duration;
87     while(1){
88         last_cycle= clock();
89         while(flag==1){
90             next_cycle = clock();
91             duration = (next_cycle - last_cycle) / (double) CLOCKS_PER_SEC;
92             //cout<<duration<<endl;
93             if(duration >= 1.5){
94                 gpioWrite(Q2, 0);
95                 gpioPWM(Q1, 0);
96                 gpioWrite(Q4, 0);
97                 gpioPWM(Q3, 0);
98                 gpioServo(SERVO,1500);
99                 cout<<endl<<"Conexão perdida..."<<endl;
100             }
101             sleep(0.25);
102         }
103         sleep(1);
104     }
105 }
106
107 void *myfunc(void *arg){
108     try{
109         UDPSocket sockrecv(SERVPORT);
110         unsigned short sourcePort;
111         string sourceAddress;
112         int servotemp=0;
113         while(1){
114             //-----
115             //recebe dados de controle do automovel
116         }
117     }
118 }
    
```

Figura 21 – Interface do programa Notepad++.

Fonte: Autoria própria.

Para que não fosse necessário usar um conjunto de monitor, teclado e mouse, utilizou-se um acesso remoto ao minicomputador através do programa PuTTY, economizando espaço e tempo. O PuTTY (Figura 22) é um software de emulação de terminal de código livre que suporta diversos protocolos de rede. Dentre esses protocolos destacamos o *Secure Shell* (SSH), destinado a suportar o acesso remoto a servidores via *shell* seguro e criptografado. O protocolo SSH foi usado neste projeto para rodar uma seção remota na *Raspberry Pi* através da rede, possibilitando gerenciar arquivos, monitorar o sistema, executar e encerrar programas via linha de comando a partir de um notebook rodando uma versão do *Windows*. O acesso é feito digitando o endereço IP e escolhendo a opção SSH no programa PuTTY, e no console aberto fazendo o *login* no sistema remoto.

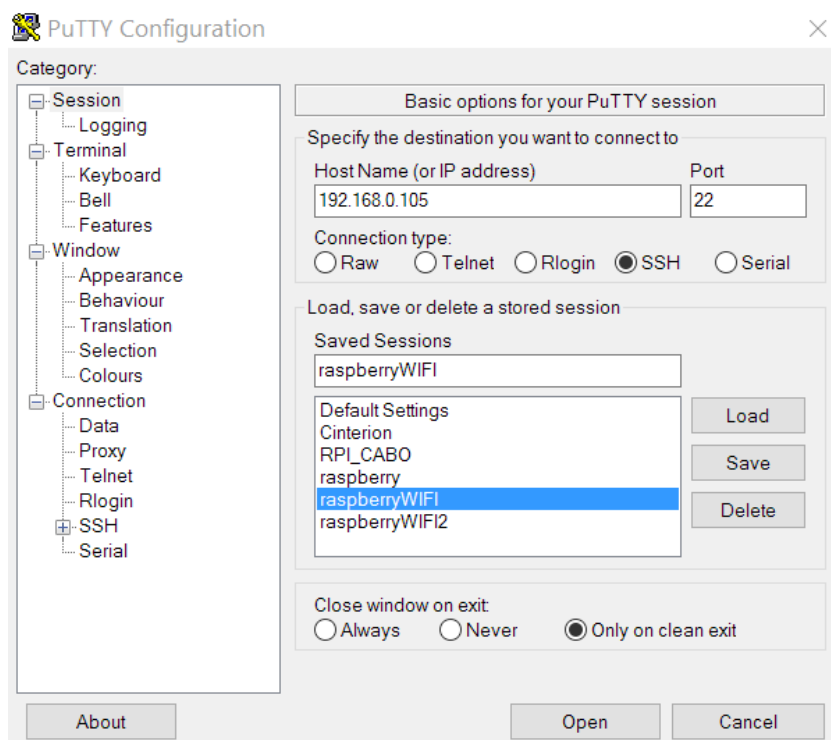


Figura 22 – Interface principal do programa PuTTY.

Fonte: Autoria própria.

Por fim, para transferir arquivos entre o notebook e a *Raspberry Pi* foi utilizado o FileZilla (Figura 23), um programa de código aberto que transfere e gerencia arquivos através do protocolo SFTP (*SSH File Transfer Protocol*). A configuração para transferir arquivos para a Raspberry é feita acessando o menu arquivo e gerenciador de sites. Após isso, aparecerá uma tela onde é possível criar um novo site digitando

o IP da *Raspberry*, o usuário, a senha e escolher o tipo de protocolo como na Figura 24.

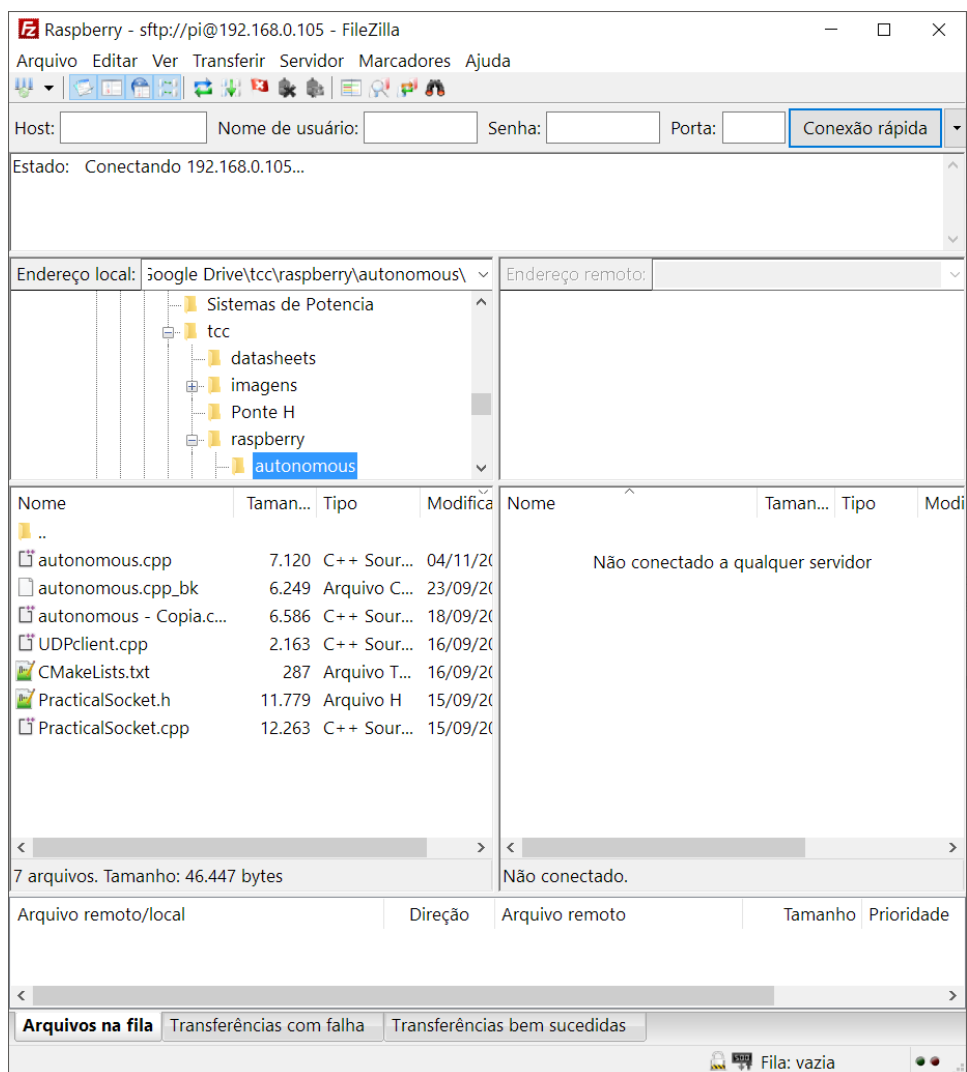


Figura 23 – Interface do programa FileZilla.

Fonte: Autoria própria.

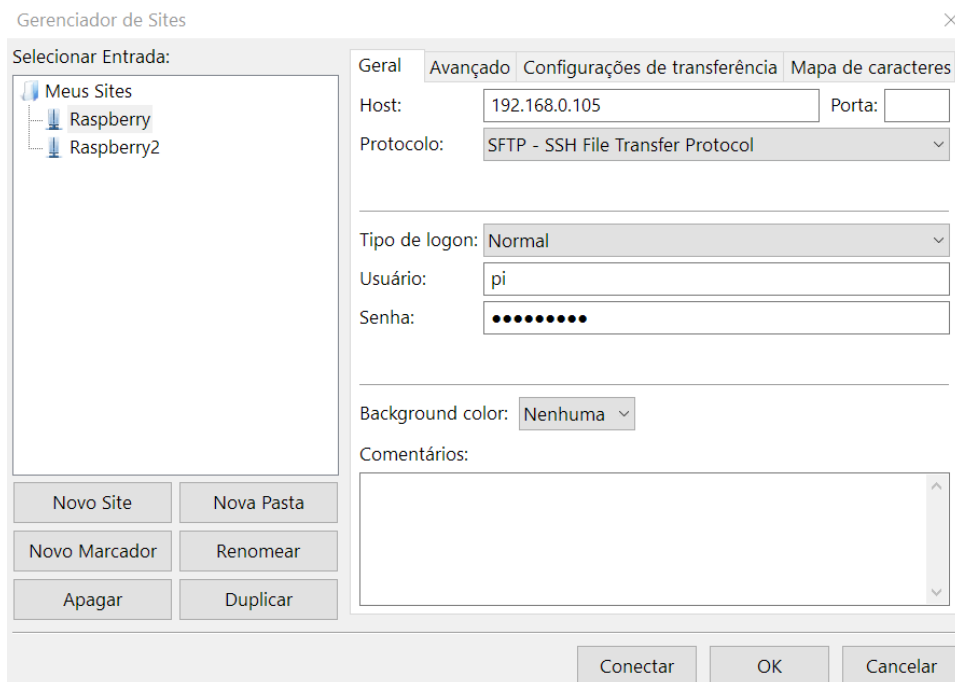


Figura 24 – Tela de criação de uma nova conexão no FileZilla.

Fonte: Autoria própria.

Para realizar a automação de compilação, foi utilizado o Cmake, que é um sistema multiplataforma compatível com sistemas Linux. O programa Cmake não gera diretamente o software final, mas sim arquivos de geração padrão (por exemplo, *makefiles* em *Unix* e projetos/espacos de trabalho no Visual C++ no *Windows*), que podem facilmente ser compilados posteriormente. O programa se baseia em um arquivo com as configurações que possui informações sobre o projeto, tais como: nome, bibliotecas e sinalizações para o processo de compilação (Figura 25).

```

1
2
3 cmake_minimum_required (VERSION 2.6)
4 project( autonomous )
5 find_package( OpenCV REQUIRED)
6 set(CMAKE_CXX_FLAGS "-Wall -ansi -pedantic -g -lnsl -lpthread -lpigpio -lm -lrt")
7 add_executable(autonomous autonomous.cpp PracticalSocket.cpp)
8 target_link_Libraries( autonomous ${OpenCV_LIBS} )

```

Figura 25 – conteúdo do arquivo CMakeLists.txt usado no projeto.

Fonte: Autoria própria.

4.2 BIBLIOTECAS USADAS

Quando se usa qualquer linguagem de programação, existe em muitos momentos a necessidade de usar algum conjunto de funções ou códigos

denominados bibliotecas. As bibliotecas são escritas previamente por outros desenvolvedores e resolvem ou auxiliam nos problemas encontrados no projeto, não sendo necessário desenvolver tudo a partir do zero. Este tópico tem como objetivo apresentar as bibliotecas de programação usadas neste projeto. São elas: OpenCV, Practical C++ Sockets, Gamepad, PiGPIO.

4.2.1 OpenCV

A OpenCV (*Open Source Computer Vision*) é uma biblioteca de código aberto para visão computacional e aprendizado de máquina. Ela foi feita para proporcionar uma infraestrutura comum para aplicações de visão computacional e para acelerar o uso de percepção de máquina em produtos comerciais. Possui suporte às linguagens de programação C/C++, Python, Java e MATLAB, e é suportado em sistemas operacionais como Windows, Linux, Android e Mac OS.

Essa biblioteca está presente nas duas plataformas neste projeto, tanto na *Raspberry Pi* instalada no sistema embarcado, quanto no *notebook* situado na estação fixa. Os módulos do OpenCV usados na implementação do projeto foram:

- **Highgui** - Usada para fazer a interface gráfica para usuário.
- **Core** - Usado para definir estruturas básicas de dados, como matrizes multidimensionais e funções básicas usadas nos outros módulos.
- **ML** - Usada para as funções de aprendizado de máquina, neste caso para as funções de redes neurais múltiplas camadas.

4.2.2 Practical C++ Sockets

A Practical C++ Sockets é uma biblioteca de código aberto para comunicação entre servidor e cliente por meios dos protocolos de redes UDP/TCP. Ele foi escrito para ser multi-plataforma, sendo suportados por sistemas baseados em Linux e Windows. Se encaixou perfeitamente para este projeto, pois pode ser usado em ambos sistemas operacionais.

4.2.3 Gamepad

A biblioteca Gamepad cria uma interface para o hardware do controle de jogos compatíveis com Microsoft DirectX e o programa escrito. Com ela foi possível conectar um controle ao programa, utilizando as funções tanto para leitura dos direcionais e botões analógicos, como também para os botões digitais. Esta biblioteca foi desenvolvida somente para ambiente *Windows*.

4.2.4 PiGPIO

Esta biblioteca tem como função implementar uma interface para controle dos pinos digitais (GPIO, entradas e saídas digitais de propósito geral) das placas *Raspberry Pi*. Tem suporte às linguagens de programação C/C++ e Python, e sua estrutura de uso é simples e intuitiva. Algumas vantagens dessa biblioteca são:

- A fácil configuração das entradas e saídas digitais, como modo de operação e modo de internos de pull-down/pull-up;
- Leitura e escrita nos pinos com apenas um comando;
- Tempo de PWM e pulsos de Servos temporizados via hardware.
- Comunicação serial via software usando qualquer GPIO;
- Amostras de GPIO temporizadas via hardware a cada 5 μ s;

4.3 PROGRAMA CRIADO PARA O SISTEMA EMBARCADO

O programa criado para *Raspberry Pi* (sistema embarcado) opera basicamente o *hardware* a ela conectado, que são a câmera, o modulo WIFI, a ponte H e o servo motor. O programa se divide em duas tarefas principais, a captação e envio de imagens geradas pela câmera para a estação fixa, e o recebimento do comando para acionamento do motor e servo motor. Uma visão geral sobre o funcionamento do programa é mostrada na Figura 26.

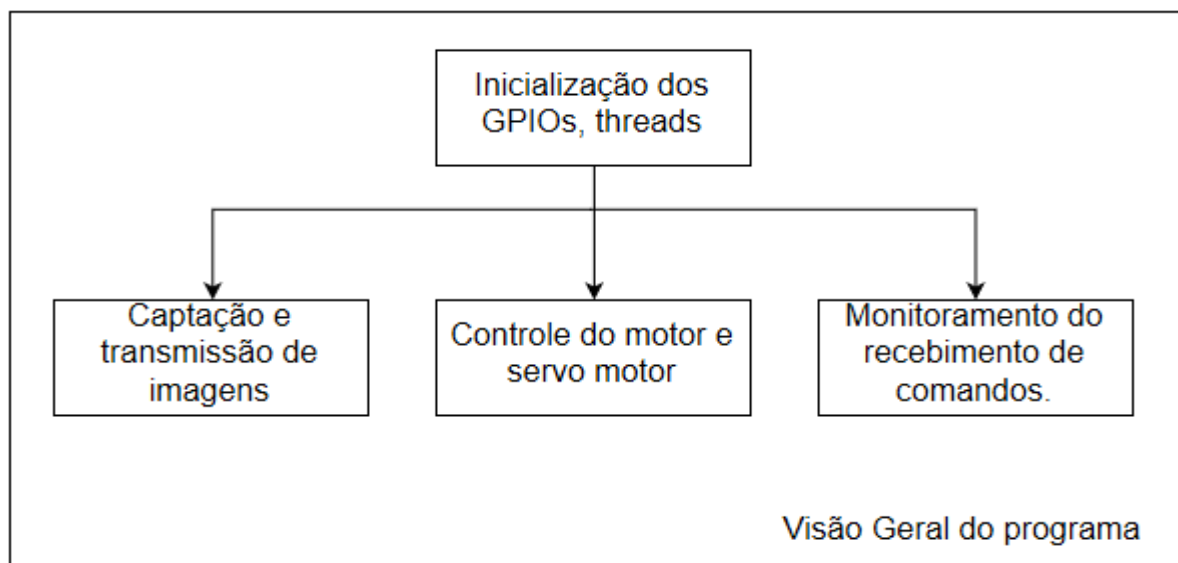


Figura 26 – Visão geral sobre o funcionamento do programa rodando na *Raspberry Pi*.

Fonte: Autoria própria.

O procedimento de captação de imagens, demonstrado no diagrama presente na Figura 27, é iniciado configurando a comunicação UDP para uma porta exclusiva para a transferência dos pacotes contendo as imagens. O algoritmo é um ciclo infinito que envolve a captação de um quadro de imagem; a escolha da região de interesse da imagem, que é a metade inferior excluindo o horizonte; a compactação da imagem; e o envio da mesma via UDP. A taxa de quadros transferidos por segundo neste projeto alcançou valores entre 20 e 25 quadros/segundo.

Para o processo de controle do motor e servo motor (Figura 28) temos uma dinâmica semelhante à de captação de imagens, iniciando com a configuração da comunicação em uma porta exclusiva para comandos, e após isso um ciclo infinito, recebendo informação referente ao controle do servo motor e do motor CC. Os dados recebidos neste caso são um vetor contendo em primeira posição um valor de 0 a 255, informando a aceleração à frente; em segunda posição, o mesmo valor porém indicando ré; e o último dado do vetor indicando a posição do servo motor, tendo 0 (zero) como posição do servo mais à esquerda, 63 posição à esquerda, 127 ao centro, 190 à direita e 255 mais à direita. Estes dados são convertidos nos formatos a serem usados para suas respectivas funções.

Uma tarefa extra foi feita também a fim de monitorar o recebimento dos comandos do processo de controle do motor e servo motor (Figura 29). Nesta tarefa é verificado o tempo entre os recebimentos dos comandos, e quando ultrapassam um valor limite os motores são desligados.

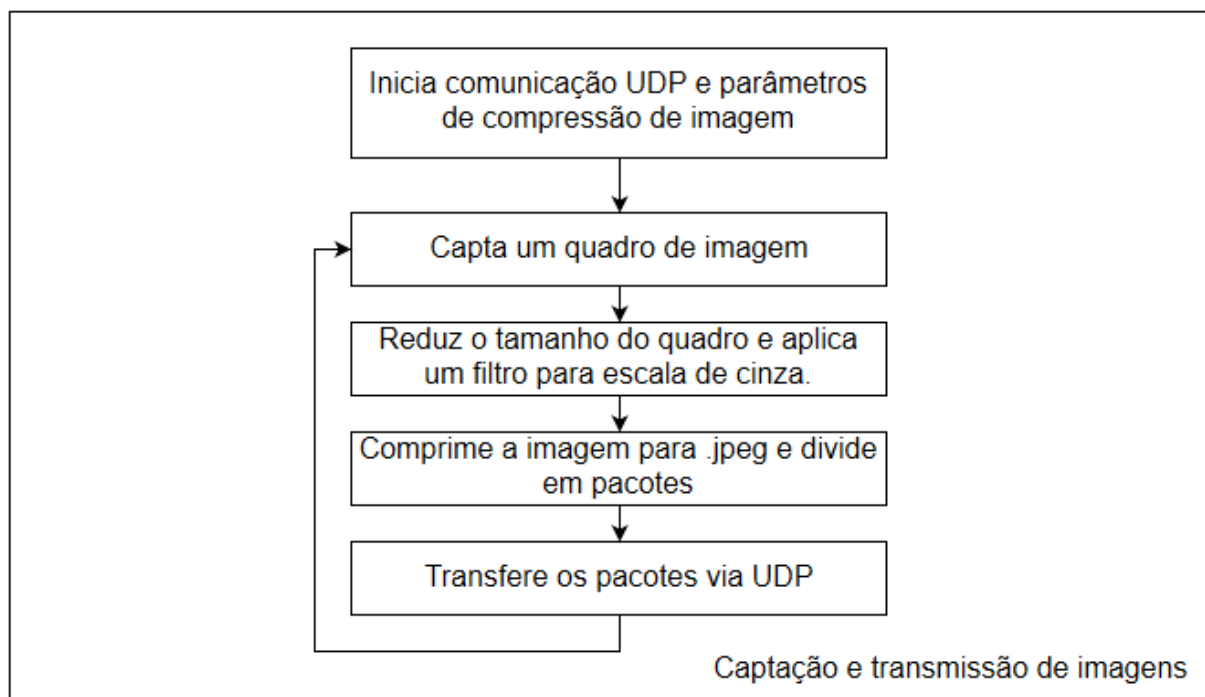


Figura 27 – Procedimentos executados na tarefa de captação e transmissão de imagens.

Fonte: Autoria própria.

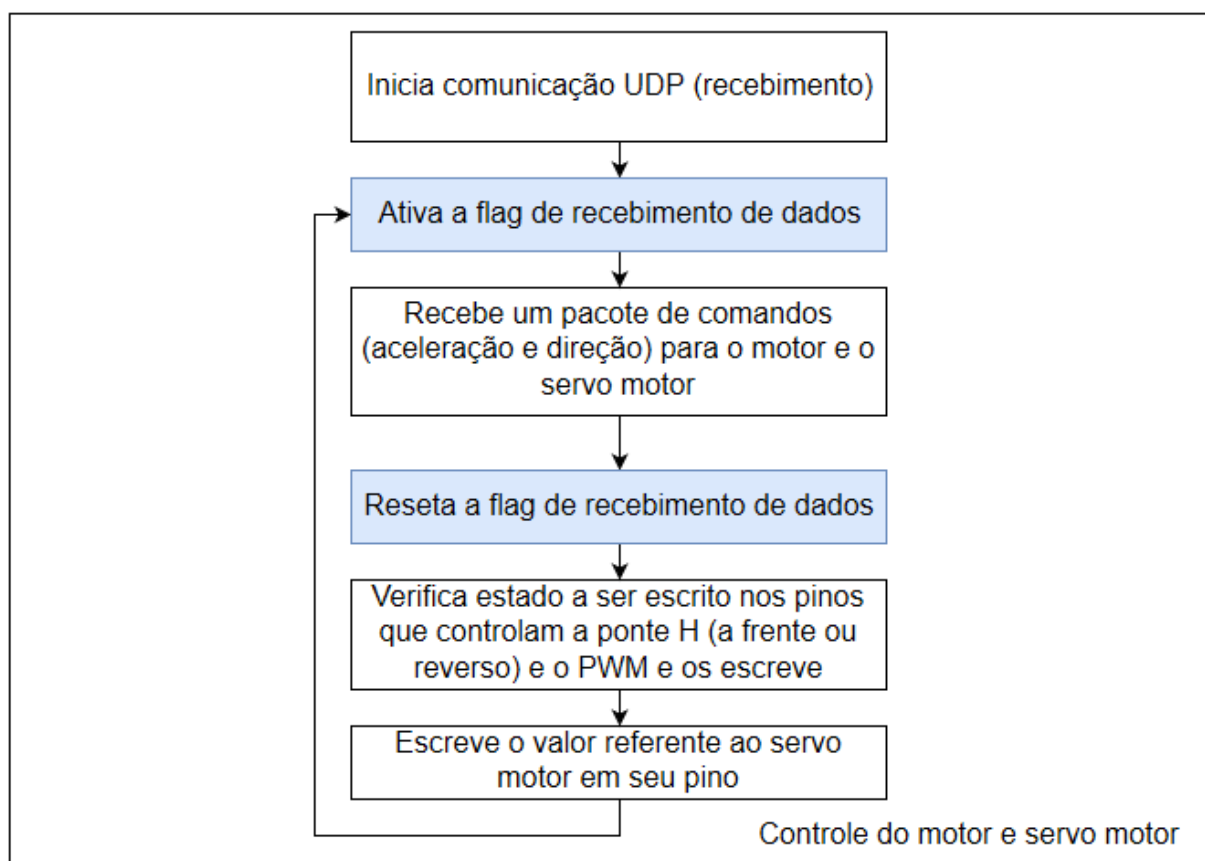


Figura 28 – Procedimentos executados na tarefa de controle do motor e servo motor.

Fonte: Autoria própria.

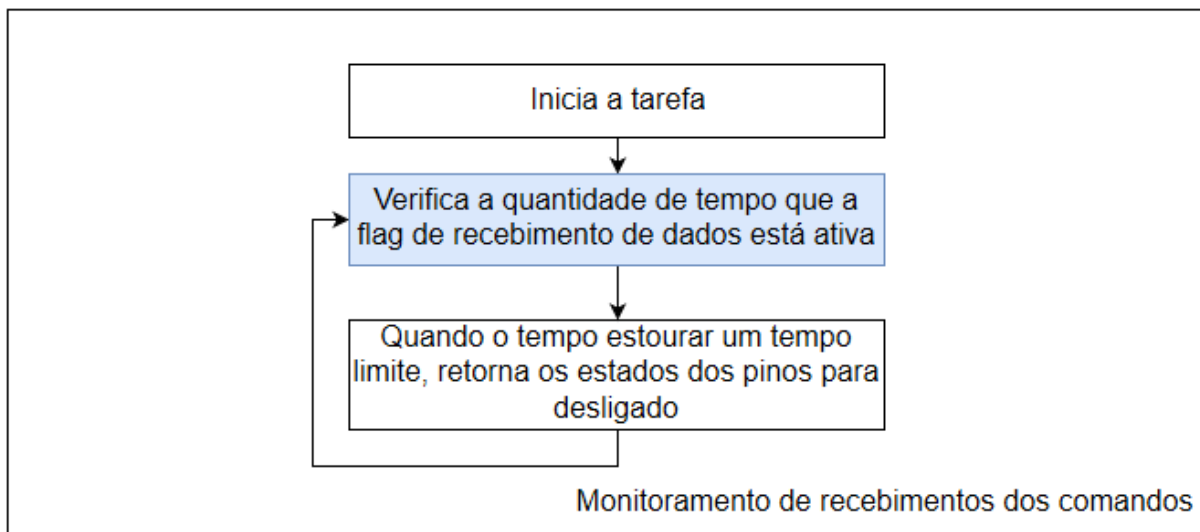


Figura 29 – Procedimentos executados na tarefa de monitoramento de recebimentos dos comandos.

Fonte: Autoria própria.

O controle dos pinos foi feito usando a biblioteca PiGPIO descrita em tópicos anteriores. Os pinos usados para este projeto estão descritos na tabela a seguir e mostrados na Figura 30.

Tabela 1 – Pinos utilizados da *Raspberry Pi*.

Nome	Função	Pino	GPIO
SERVO	Servo	29	26
Q1	PWM	31	5
Q2	Saída digital	33	6
Q3	PWM	35	13
Q4	Saída digital	37	19
GND	GND	39	-

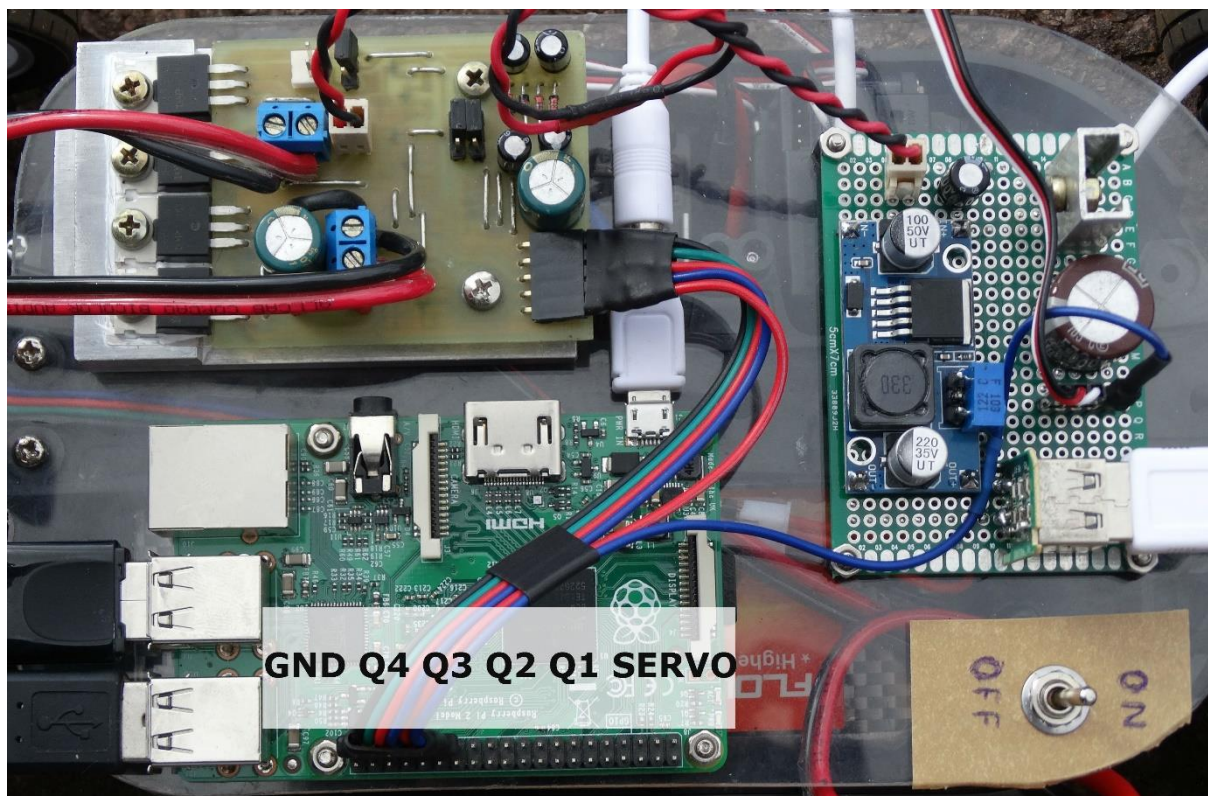


Figura 30 – Ligação dos pinos *Raspberry Pi* para outros periféricos.

Fonte: Autoria própria.

4.4 PROGRAMA CRIADO PARA A ESTAÇÃO FIXA

O programa principal, escrito para o computador, possui diferentes modos de execução tais como coleta de dados, treinamento da rede neural, execução da rede neural, execução manual, e teste de confiabilidade da rede (Figura 31). Esses modos são selecionáveis ao executar o programa.

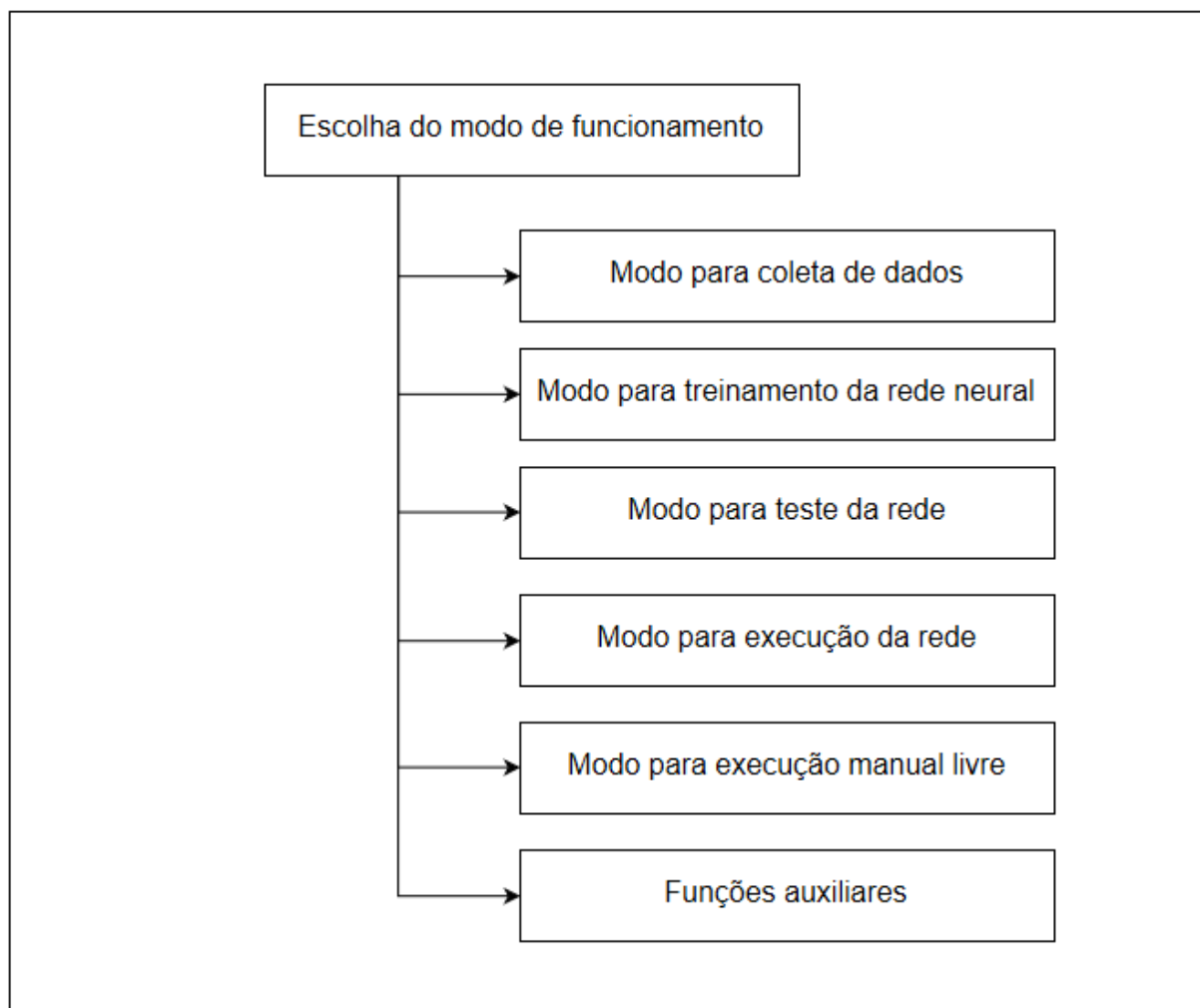


Figura 31 – Visão geral sobre o programa rodando no computador.

Fonte: Autoria própria.

4.4.1 Coleta de dados

O funcionamento da coleta de dados pode ser observado no diagrama presente na Figura 32. Este procedimento utiliza as bibliotecas Gamepad, Practical C++ Sockets e OpenCV. Para este modo foi usado duas tarefas em paralelo, uma para leitura do controle joystick e envio dos comandos lidos via UDP para a *Raspberry PI*, e outra recebendo e tratando os quadros de imagem. Ao final deste procedimento tem-se um conjunto de dados contendo as imagens que serão usadas para o treinamento da rede, as imagens que serão usadas para teste de convergência da rede, e os dados de aceleração e direção aplicadas no instante em que as imagens

foram recebidas. Além disso, é obtida também essas imagens em formato .jpeg possibilitando a sua visualização.

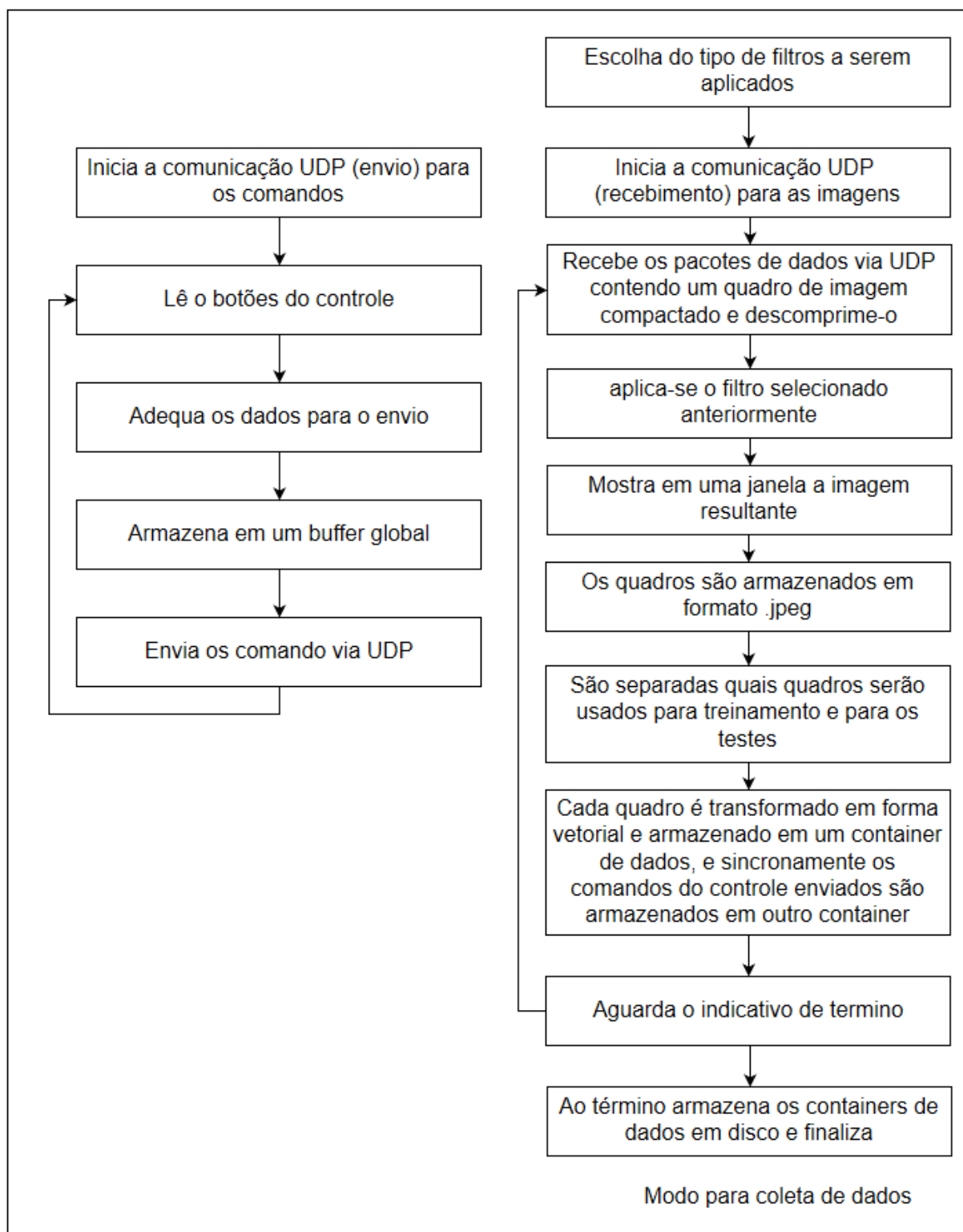


Figura 32 - Procedimentos executados no modo para coletas de dados.

Fonte: Autoria própria.

4.4.2 Treinamento

Em modo treinamento (como descrito na Figura 33) os dados para são carregados e os parâmetros para a rede neural definidos: quantidade de camadas, número de neurônios em cada camada, número máximo de iterações, erro, taxa de aprendizado e momentum. O módulo de aprendizado de máquina da biblioteca OpenCV foi usado neste modo.

A quantidade de entradas e saídas são fixados pelos dados captados na coleta dependendo da quantidade de pixels e da quantidade de saídas possíveis. Neste projeto foram realizados treinamentos para uma e duas camadas ocultas com os seguintes parâmetros apresentados na Tabela 1.

Tabela 1 - Parâmetros usados para treinamento da rede neural.

PARÂMETRO	VALOR
Quantidade máxima de iterações	500
Épsilon – Erro	0.001
Critério de parada	Número de ciclos e erro
Treinamento	Backpropagation
Taxa de aprendizado	0.01
Momentum	0.01

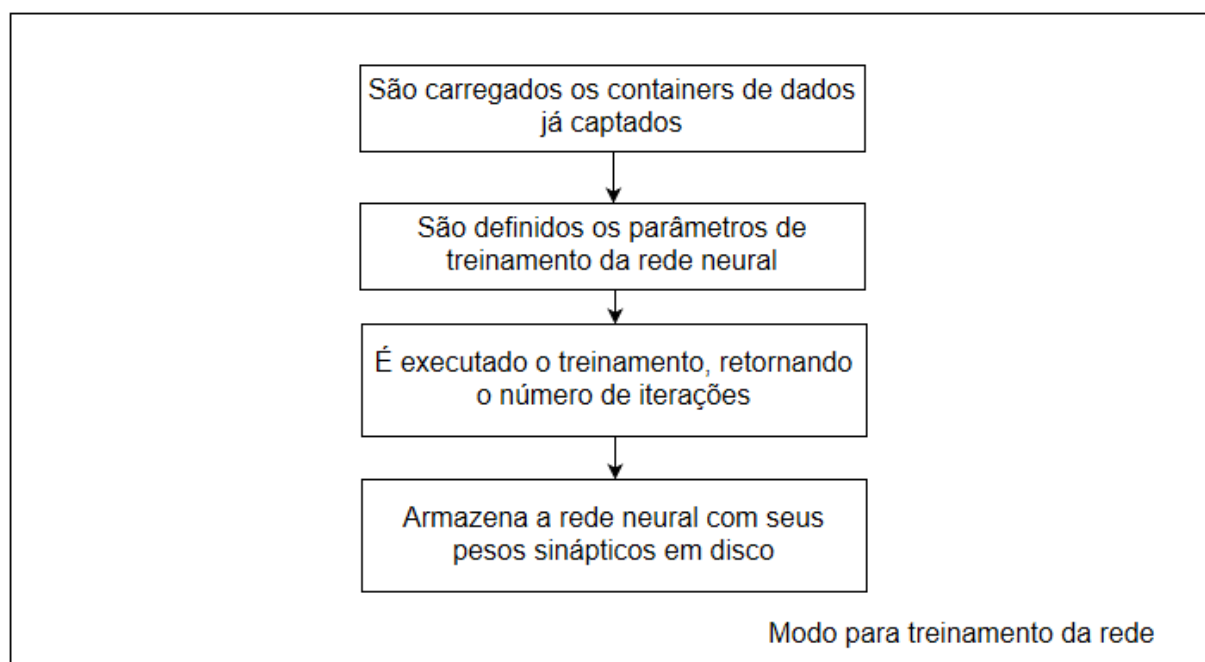


Figura 33 - Procedimentos executados no modo para treinamento da rede.

Fonte: Autoria própria.

4.4.3 Testes da rede

Para a validação da rede treinada foi desenvolvido um medidor de confiabilidade baseado na quantidade de acertos provenientes dos dados para os quais a rede não foi treinada. Esses dados nós chamamos de dados para teste, e nesse procedimento (Figura 34) esses dados são carregados e aplicados a rede neural treinada, gerando uma predição. A predição para cada quadro de imagem é comparada com a saída esperada, sendo computado cada acerto. Ao final é calculado em percentual a quantidade de acerto em relação a quantidade de dados de teste.

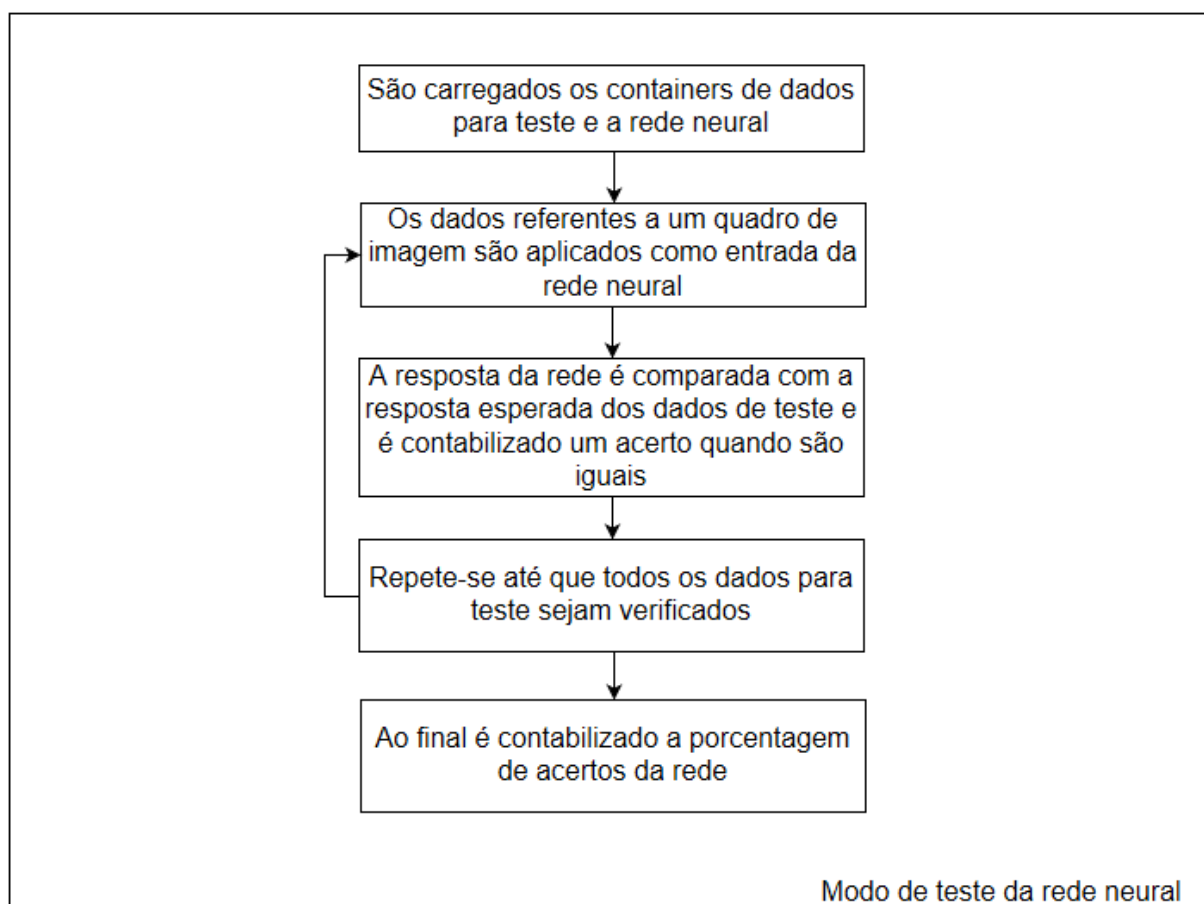


Figura 34 - Procedimentos executados no modo para teste da rede neural.

Fonte: Autoria própria.

4.4.4 Execução da rede neural

A descrição do modo execução da rede pode ser observado no diagrama presente na Figura 35. Este procedimento se utiliza das bibliotecas Gamepad, Practical C++ Sockets e OpenCV. Para esse modo não foi usado uma tarefa paralela para a leitura do joystick.

O controle *joystick* alterna o modo de operação entre manual e automático. Ao entrar em modo automático, os quadros de imagens captados são aplicados à rede neural resultando em predições, que após serem transformadas ao formato correto é enviado ao sistema embarcado. A função responsável por calcular uma predição da rede a partir de uma imagem pode fazê-la em menos de 2 milissegundos em casos de duas camadas ocultas, e menos de 1 milissegundo para uma camada oculta. O tempo desde o envio até o cálculo da predição não pode ser precisamente mensurado, pois depende de várias etapas em *hardwares* diferentes.

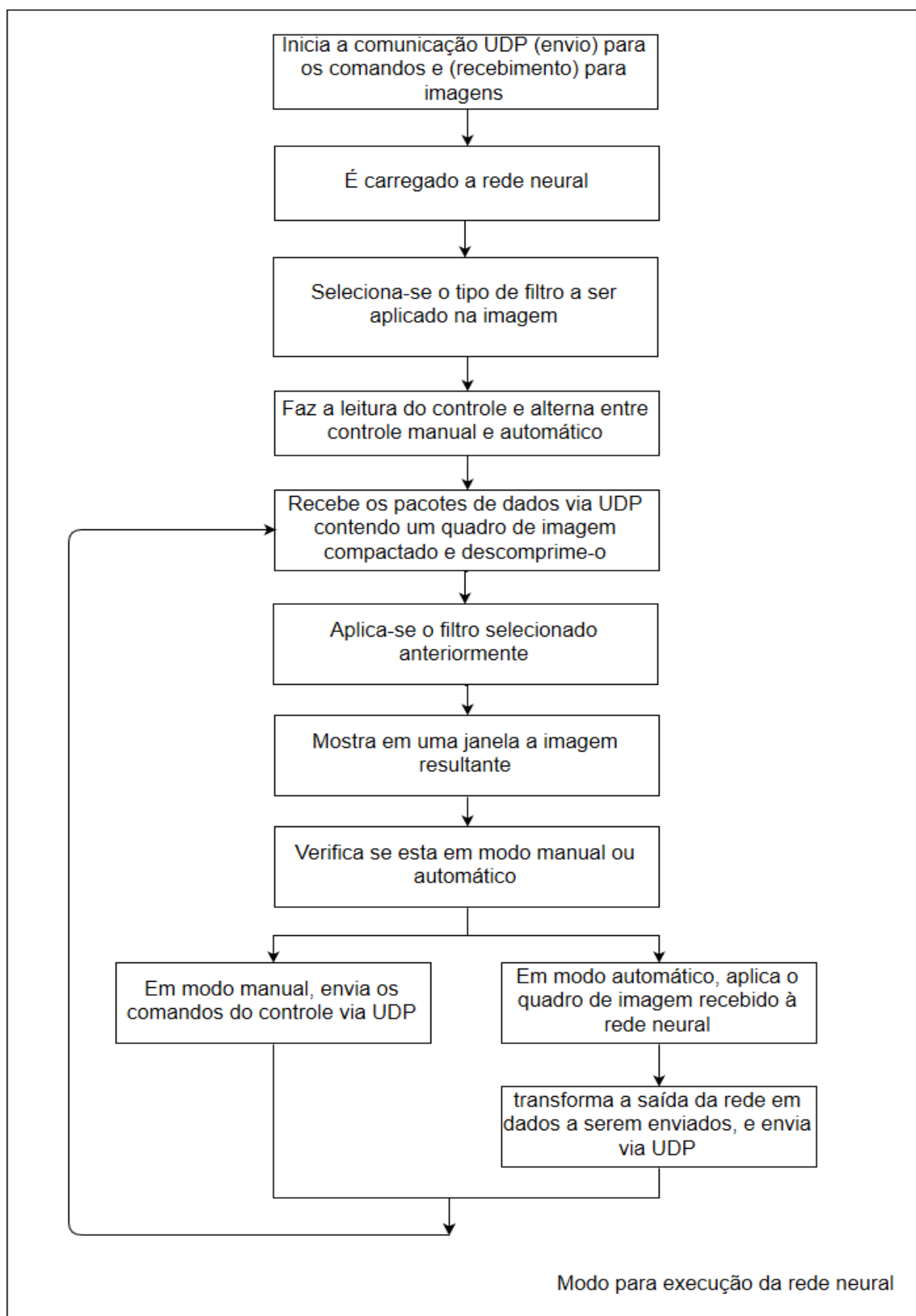


Figura 35 – Procedimentos para modo de execução da rede neural.

Fonte: Autoria própria.

4.4.5 Filtros usados nas imagens

Como citado nas seções anteriores, alguns filtros são aplicados às imagens captadas antes de serem armazenadas para o treinamento da rede neural ou aplicadas à rede neural para gerar previsões. Os filtros usados para este trabalho são: escala de cinza (Figura 37), limiar (*threshold*, Figura 38) e detecção de faixa (Figura 39).

A biblioteca OpenCV fornece funções com as quais podemos aplicar esses filtros facilmente com exceção do filtro detecção de faixa, que foi desenvolvido usando um filtro pronto em conjunto com um algoritmo próprio.

O filtro detecção de faixa foi feito aplicando à imagem, recebida pelo computador, primeiramente um filtro de desfoque gaussiano e posteriormente um filtro de detecção de bordas (*Canny*), ambos presentes na OpenCV, resultando em uma imagem preta com linhas brancas onde originalmente havia diferenças abruptas de tons de cinza. Após isso, a imagem passa pelo algoritmo criado especialmente para este projeto, capaz de colorir entre duas linhas com distancias próximas, resultando em uma imagem onde as faixas da pista estão totalmente brancas enquanto todo o resto está preto. Um exemplo da aplicação deste filtro é mostrado na Figura 36.

O filtro de escala de cinza é aplicado ainda na *Raspberry Pi*, antes de ser enviada, reduzindo a quantidade de informação transmitida via rede, agilizando assim este processo.

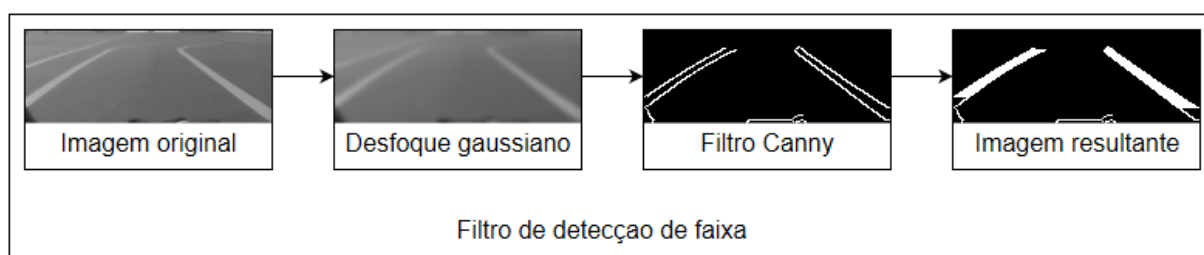
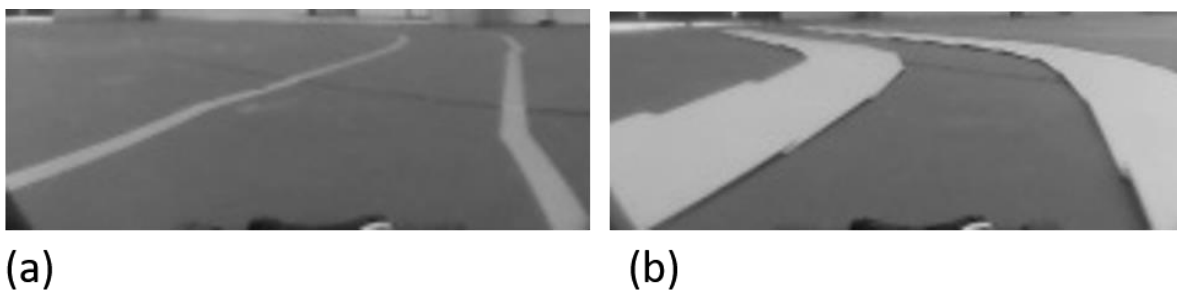


Figura 36 – Procedimentos para o filtro detecção de faixa.

Fonte: Autoria própria.

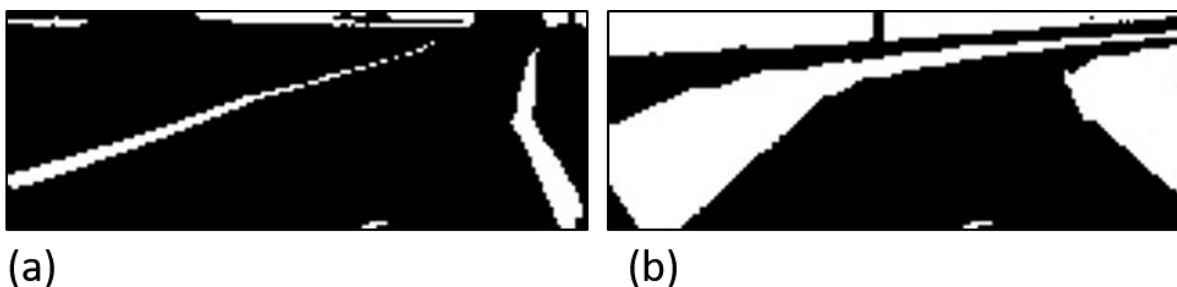


(a)

(b)

Figura 37 – Exemplo de imagem captada pelo sistema, filtro escala de cinza aplicado. (a) pista com fita adesiva, (b) fita com placas de papelão.

Fonte: Autoria própria.



(a)

(b)

Figura 38 - Exemplo de imagem captada pelo sistema, filtro limiar aplicado. (a) pista com fita adesiva, (b) fita com placas de papelão.

Fonte: Autoria própria.

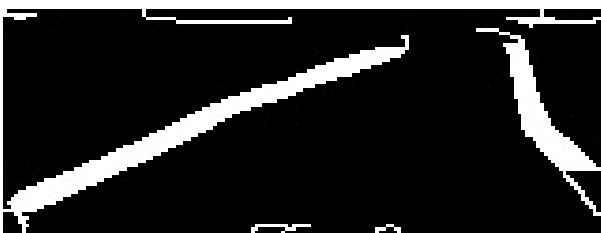


Figura 39 – Exemplo de imagem captada pelo sistema, filtro detecção de borda aplicado.

Fonte: Autoria própria.

5 TESTES E RESULTADOS

Para avaliar as RNAs foi seguido uma metodologia em relação à coleta, treino e execução das mesmas para que pudéssemos obter um valor indicativo sobre a sua eficiência. Em modo de coleta, obtive-se dados referentes a um percurso realizado, esses dados foram divididos, onde 75% deles foram utilizados para o treinamento das redes e os outros 25% foram utilizados para teste e obtenção de um valor percentual de acertos (a cada três dados coletados, um seria destinado ao teste).

Após o treinamento, foram aplicadas à cada RNA seus respectivos dados de teste, e para cada imagem de teste aplicada à rede, comparou-se os valores de saída da rede com o esperado. A comparação se deu da seguinte maneira:

1. Foi verificado qual era a posição do maior valor dentre as cinco saídas da rede;
2. Foi verificado se esse valor era maior que 0,5;
3. Foi comparado se essa posição da saída da rede era a mesma que a esperada (coletada) e se sim, essa condição indicaria um acerto da RNA;
4. Foi aplicado todos os dados de teste e contado quantos acertos a RNA teve;
5. Foi realizado o cálculo da porcentagem de acertos em relação a quantidade de dados usados para teste;
6. Utilizou-se esse valor como indicativo de confiabilidade da rede.

No total foram treinadas 21 RNAs com diferentes tamanhos (número de camadas ocultas e neurônios), divididas entre os diferentes tipos de pista, diferentes filtros aplicados à imagem. Os resultados estão descritos nas Tabelas 2 e 3.

Tabela 2 – Resultados das redes neurais para pista de papelão.

Pista larga montada com placas de papelão					
Filtro de imagem usado	Quantidade de camadas da rede neural	Número de neurônios nas camadas ocultas	Quantidade de dados captados para treinamento	Quantidade de dados captados para testes	Porcentagem de acertos
Limiar	3	5			67%
	3	32	245	61	88%
	4	32 e 5			93%
Escala de cinza	3	5			66%
	3	32	250	62	45%
	4	32 e 5			80%
Pista estreita montada com placas de papelão					
Filtro de imagem usado	Quantidade de camadas da rede neural	Número de neurônios nas camadas ocultas	Quantidade de dados captados para treinamento	Quantidade de dados captados para testes	Porcentagem de acertos
Limiar	3	5			64%
	3	32	234	58	67%
	4	32 e 5			23%
Escala de cinza	3	5			60%
	3	32	237	59	44%
	4	32 e 5			48%

Tabela 3 - Resultados das redes neurais para pista de fita adesiva

Pista montada com fita adesiva					
Filtro de imagem usado	Quantidade de camadas da rede neural	Número de neurônios nas camadas ocultas	Quantidade de dados captados para treinamento	Quantidade de dados captados para testes	Porcentagem de acertos
Limiar	3	5			73%
	3	32	619	155	92%
	4	32 e 5			60%
Escala de cinza	3	5			80%
	3	32	340	85	61%
	4	32 e 5			32%
Detecção de faixa	3	5			81%
	3	32	609	152	94%
	4	32 e 5			94%

Como pode ser observado na Tabela 2, o veículo na pista estreita (aproximadamente 30 cm) não teve um bom desempenho chegando a apenas 67 % de acerto com o filtro limiar. Já para uma pista mais larga (aproximadamente 45 cm) conseguiu-se um resultado de 93% de acerto para o mesmo tipo de filtro. Acreditamos que esta diferença se deve ao fato do controle manual sobre a pista estreita ser mais difícil, tornando a captação dos dados muito variante e gerando, para imagens semelhantes, valores esperados muito diferentes. Além disso, para a pista de papelão, o filtro de imagem detecção de faixa não foi aplicado pois a sua criação foi pensada exclusivamente para a pista com fita, não gerando imagens satisfatórias ao ser aplicado à pista feita com placas de papelão.

Na Tabela 3 é possível observar que o maior percentual de acertos com a pista de fita adesiva ocorreu com o filtro de imagem de detecção de faixa, atingindo 94% de acerto nos testes.

Após obter os percentuais de acerto, foi feita uma avaliação prática onde foi colocado o automodelo para rodar cada RNA. Nesse teste, podemos verificar um bom desempenho da RNA para a pista de papelão com filtro de imagem limiar (semelhante ao teste apresentado nas Tabelas 2), onde a mesma executou várias vezes os percursos de ida e volta na pista sem cometer graves erros. Para essa RNA ocorreu alguns erros quando tentou aumentar a velocidade do veículo, que em certas partes da pista acabava ganhando embalo devido a irregularidades na mesma.

Houve também, um desempenho considerável da RNA para a pista de fita e imagem com preenchimento de borda e limiar (validando assim, a etapa de testes mostrado na Tabela 3), a qual executou boa parte da pista, tendo errado em certos pontos da mesma onde houve uma captação inadequada de imagem, as vezes com falta de identificação da borda outras vezes com captação de ruídos.

As outras redes foram testadas, mas sem obter um desempenho muito bom, com acertos em algumas partes das pistas e erros em outras. Nessa observação do teste prático, viu-se que as redes que apresentaram melhores desempenhos foram as que obtiveram o índice de acerto maior.

6 CONSIDERAÇÕES FINAIS

Este trabalho visou desenvolver um projeto com materiais fáceis de serem encontrados, integrados a técnicas de processamento e controle, podendo assim verificar que é possível desenvolver pesquisas relacionadas a processamento de imagem e autonomia de modelos. Neste capítulo serão apresentadas algumas observações finais, bem como propostas para trabalhos futuros.

6.1 CONCLUSÃO

Com os resultados apresentados, pode-se concluir que o presente trabalho cumpriu com os objetivos de projetar um veículo que consiga trafegar de maneira autônoma utilizando RNA para tomar as decisões a partir das imagens captadas. Além disso, o sistema embarcado com o minicomputador mostrou ter um bom desempenho, assim como os materiais utilizados, o processamento de imagem, a troca de dados via rede, e a utilização de programas e bibliotecas próprias para a aplicação de RNAs.

Com esse sistema, foi seguido uma metodologia para a coleta de dados e treinamento de rede, onde foram utilizados vários tipos de processamento de imagem em duas pistas diferentes que obtiveram resultados diferentes na prática. Após o treinamento das RNAs, foi realizado a validação das mesmas através do método descrito anteriormente, obtendo um valor percentual de acertos e de testes práticos nos quais o veículo conseguiu realizar certos percursos de pista, tendo resultados negativos em poucas ocasiões (geralmente geradas pela detecção deficitária da pista).

Analisando todo o projeto e seus resultados, podemos considerar que as RNAs demonstraram uma considerável aptidão para aprender sobre o controle de direção utilizando imagens como entrada. Com isso, o sistema embarcado criado possibilitou a realização de coleta de dados, treinos e testes em geral (manuais e de RNAs) de uma maneira boa para o desenvolvimento de um projeto na área de veículos autônomos, podendo com algumas melhorias, continuar sendo usado em possíveis trabalhos futuros.

6.2 SUGESTÕES PARA TRABALHOS FUTUROS

O presente trabalho teve foco apenas no controle de direção de um veículo autônomo, tendo assim na parte de aceleração, utilizado apenas valores pré-definidos. Contudo, na execução das RNAs, mesmo utilizando apenas um único valor de aceleração para o veículo, o mesmo apresentou velocidades diferentes em determinadas partes da pista, conforme irregularidades no nível da mesma. Considerando esse fato, tem se como sugestão a implementação de um sistema para manter a velocidade constante ou dentro de uma certa faixa, na qual o veículo poderá processar o controle de direção sem se perder ou sem andar muito devagar. Para isso seria necessário o uso da velocidade atual do automodelo como dado de entrada desse controle.

Outra sugestão se refere ao fato de criar mais níveis para a direção, algo que seria interessante principalmente se pudesse ser feito mais treinamentos das RNAs com variados tipos de curva (no presente trabalho a pista usada tinha apenas 2 curvas). Para isso seria de grande importância obter um local mais adequado para a criação de pistas e curvas, pois houve grande dificuldade em localizar um ambiente propício para a criação de uma pista e realização de testes.

Outras melhorias poderiam ser feitas na área de processamento de imagem, como por exemplo, threshold dinâmico, ajustes automáticos de claridade/brilho e outras técnicas que resultem em uma melhor identificação da pista, pois pôde se perceber o quão importante é para as RNAs criadas receberem imagens adequadas e sem ruídos.

Há também o interesse da substituição do computador como acelerador de Hardware, onde poderia se testar se a própria *Raspberry Pi* conseguiria executar a RNA, ou em um caso mais ambicioso, implementar uma FPGA para a realização dos cálculos, deixando assim o protótipo com um sistema totalmente embarcado.

REFERÊNCIAS

- 1 COVINGTON, William (Coord.). **Autonomous Vehicles. Technical and Legal Challenges: An Overview of the State of the Art in Autonomous Vehicle Technology and Policy**. Autonomous Vehicles Team of the University of Washington School of Law, Clinic on Technology, Law and Public Policy 2013.
- 2 GOOGLE inc. **Google Self-driving Project**. Disponível em: <<https://www.google.com/selfdrivingcar/how/>> Acesso em 18 de maio de 2016.
- 3 MERCEDES-BENZ. **Safety: Decades of preparation. Milliseconds of proof**. Disponível em: <<http://www.mbusa.com/mercedes/benz/safety#module-4>> Acesso em 18 de maio de 2016.
- 4 TESLA MOTORS. **Performance and safety refined**. Disponível em: <https://www.teslamotors.com/en_EU/models> Acesso em 18 de maio de 2016.
- 5 WORLD HEALTH ORGANIZATION. **The top 10 causes of death**. Disponível em: <<http://who.int/mediacentre/factsheets/fs310/en/>> Acesso em 18 de maio de 2016.
- 6 LUTIN, Jerome M.; KORNHAUSER, Alain L.; LERNER-LAM, Eva. The Revolutionary Development of Self-Driving Vehicles and Implications for the Transportation Engineering Profession. **Journal - Institute of Transportation Engineers - ITE**, Washington, DC, p. 28-32, Julho de 2013.
- 7 USA. National Highway Traffic Safety Administration. **NHTSA's review of the National Automotive Sampling System: Report to Congress**. (Report No. DOT HS 812 128). Washington, DC. 47 p. Julho de 2008.

8 HAYKIN, Simon S. **Redes neurais: Princípios e prática**. Trad. Paulo Martins Engel. 2ed. Porto Alegre. Bookman, 2001.

9 KOVÁCS, Zsolt L. **Redes neurais artificiais: Fundamentos e Aplicações**. 4ed. São Paulo, Editora Livraria da Física, 2002.

10 SILVA, Eugênio; OLIVEIRA, Anderson Canêdo de. **Dicas para a Configuração de Redes Neurais**. Rio de Janeiro, 2001.

11 COSTA E SILVA, Luiz Fernando. **Modelo de Rede Neural Artificial Treinada com o Algoritmo Backpropagation**. Juiz de Fora, 2003.

12 FONTANÁRIO, Rogerio Luis; TONETE, Jeu. **Códigos - Aplicação De Redes Neurais Multicamadas Para Controle De Um Veículo Autônomo**. Disponível em: <<https://github.com/jtonete/Autonomous>> Acesso em 17 de novembro de 2017.

ANEXO A – PONTE H

